

Fourier Transforms

Based on lectures by Nils Halversson at CU Boulder

Importance of FTs

- Antenna theory:
 - The far-field beam pattern is the FT of the antenna illumination pattern
- Optics:
 - An FT relation exists between back and front focal points of a converging lens. For example: a plane wave is transformed into a delta function. A lens is therefore a Fourier transform device.
- Quantum mechanics
 - There is an FT relation between a particles position $\vec{x}$ and its momentum $\vec{p}$
- Probability
 - If x, y are independent random variables, if we define a new random variable $z = x + y$, then the PDF of z is $f_z(z) = \int_{-\infty}^{\infty} dx f_x(x) f_y(z - x)$
 - In otherwords, the PDF of a sum of two random variables is a convolution
- Interferometry
 - The output of an interferometer is the FT of the sky intensity distribution.
- Signal processing and filtering
 - Much of filtering is done in the (spatial or temporal) frequency domain.
- Spectroscopy
 - Many types of spectrometer: grating, Fabry-Perot, Fourier transform spectrometer (FTS), work on the principle of coherent wave interference, which can be understood with Fourier transforms.

```
In [1]: %matplotlib inline
import pylab as pl
pl.style.use('dark_background')
pl.rcParams['font.size'] = 14
pl.rcParams['image.origin'] = 'lower'
pl.rcParams['figure.dpi'] = 100
```

```
In [2]: import numpy as np
```

In the following examples, we will use a Fourier transform pair of t , time, and ν , frequency, where dimensionally, $[t] = [1/\nu]$ (i.e., $[t] = \text{s}$ and $\nu = [\text{Hz}]$)

We will sometimes also relate space and spatial frequency (or wavenumber), often denoted x and k .

Definition:

The Fourier transform $F(t)$ of a function $f(\nu)$, also written $FT\{f(\nu)\}$, is

$$FT\{f(\nu)\} = F(t) = \int_{-\infty}^{\infty} f(\nu) e^{-2\pi i \nu t} d\nu$$

giving an inverse

$$FT\{F(t)\} = f(\nu) = \int_{-\infty}^{\infty} F(t) e^{+2\pi i \nu t} dt$$

Proof that these are inverses

Exercise:

plug one into the other and prove you get an equality

Simple Fourier Transforms

Sines and cosines:

If $f(\nu) = \cos \nu$, then the Fourier transform of f is:

$$F(t) = \int_{-\infty}^{\infty} (\cos \nu) e^{-2\pi i \nu t} d\nu$$

Using Euler's formula

$$e^{i\nu} = \cos \nu + i \sin \nu$$

$$e^{-i\nu} = \cos \nu - i \sin \nu$$

we get $\cos \nu$:

$$\frac{1}{2}(e^{i\nu} + e^{-i\nu}) = \cos \nu$$

We can substitute our expression for $\cos \nu = \frac{1}{2}(e^{i\nu} + e^{-i\nu})$ into

$$F(t) = \int_{-\infty}^{\infty} (\cos \nu) e^{-2\pi i \nu t} d\nu$$

to get

$$F(t) = \int_{-\infty}^{\infty} \frac{1}{2}(e^{i\nu} + e^{-i\nu}) e^{-2\pi i \nu t} d\nu$$

Combining terms, we get:

$$F(t) = \int_{-\infty}^{\infty} \frac{1}{2}(e^{i\nu-2\pi i \nu t} + e^{-i\nu-2\pi i \nu t}) d\nu$$

Which, with a little more manipulation (pulling out $i\nu$), becomes:

$$F(t) = \int_{-\infty}^{\infty} \frac{1}{2} \left(e^{i\nu(1-2\pi t)} + e^{-i\nu(1+2\pi t)} \right) d\nu$$

We can inspect this equation a bit to see what it tells us.

$$F(t) = \int_{-\infty}^{\infty} \frac{1}{2} \left(e^{i\nu(1-2\pi t)} + e^{-i\nu(1+2\pi t)} \right) d\nu$$

If $t = 1/2\pi$, the first term becomes $e^0 = 1$, so the integral is an infinite integral of a positive constant - it goes to positive infinity.

If $t = -1/2\pi$, the second term is positive.

For *any other* value of t , there is still an imaginary (oscillating) component, so the infinite integral is zero.

This integral $F(t) = \int_{-\infty}^{\infty} (\cos \nu) e^{-2i\pi t\nu} d\nu$ is zero *except* when $1 - 2\pi t = 0$ or $1 + 2\pi t = 0$.

$$F(t) = \int_{-\infty}^{\infty} (\cos \nu) e^{-2i\pi t\nu} d\nu$$

$F(t) = 0$ unless $t = \frac{1}{2\pi}$ or $t = -\frac{1}{2\pi}$, in which case it is ∞

$$F(t) = FT\{\cos \nu\} = \delta\left(t \pm \frac{1}{2\pi}\right)$$

δ is the [Dirac Delta](#)

The Dirac Delta

You need to know these properties:

$$\delta(x) \simeq \begin{cases} +\infty, & x = 0 \\ 0, & x \neq 0 \end{cases}$$

$$\int_{-\infty}^{\infty} \delta(x) dx = 1$$

$$\int_{-\infty}^{\infty} f(t) \delta(t - T) dt = f(T)$$

The integral over *all* values of any function times the Dirac delta is just that function evaluated at a single value.

$$\int_{-\infty}^{\infty} f(t) \delta(t - T) dt = f(T)$$

In other words, the Dirac delta turns an infinite integral into a function evaluation.

Fourier Transform Pairs

Rather than derive more solutions, we'll show what some solutions look like.

We'll plot the real and imaginary components of the plots separately, and we'll also plot the *norm*, aka the absolute value:

$$||x||^2 = \text{real}(x)^2 + \text{imag}(x)^2$$

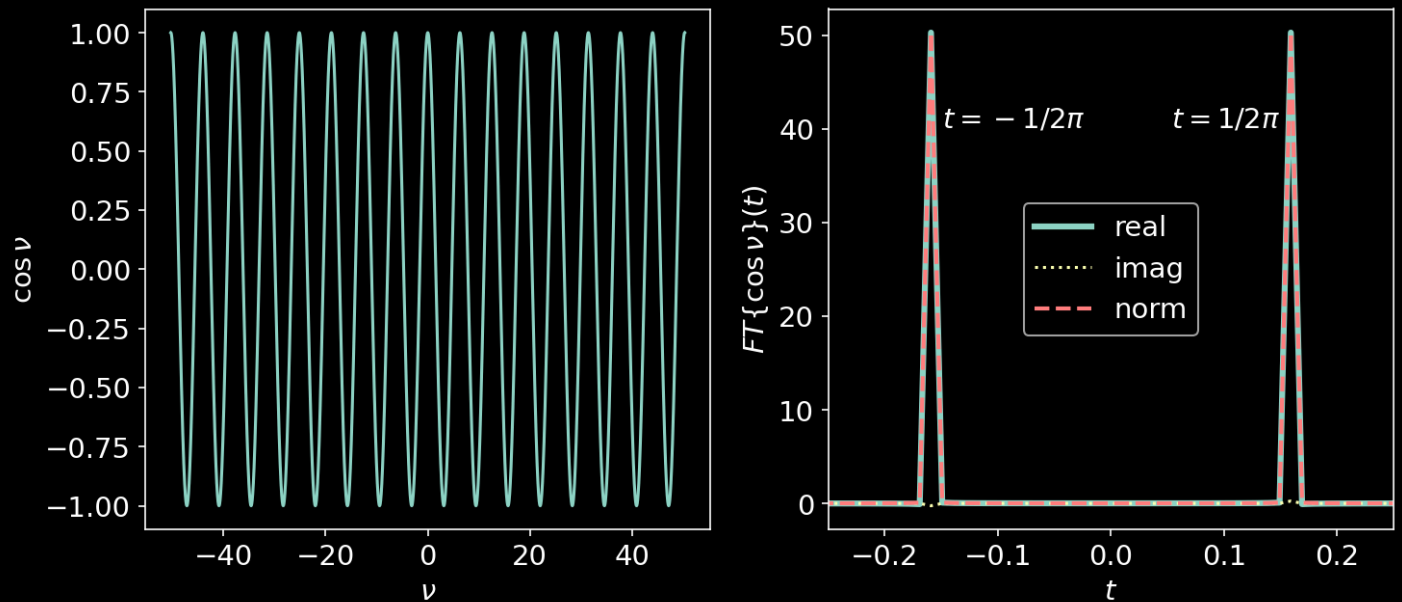
Start with cosine.

The pair is $FT\{\cos \nu/\nu_0\} = \delta\left(t \pm \frac{1}{2\pi\nu_0}\right)$.

When evaluating the δ in a discrete Fourier transform (i.e., on actual data), we draw it as a single spike.

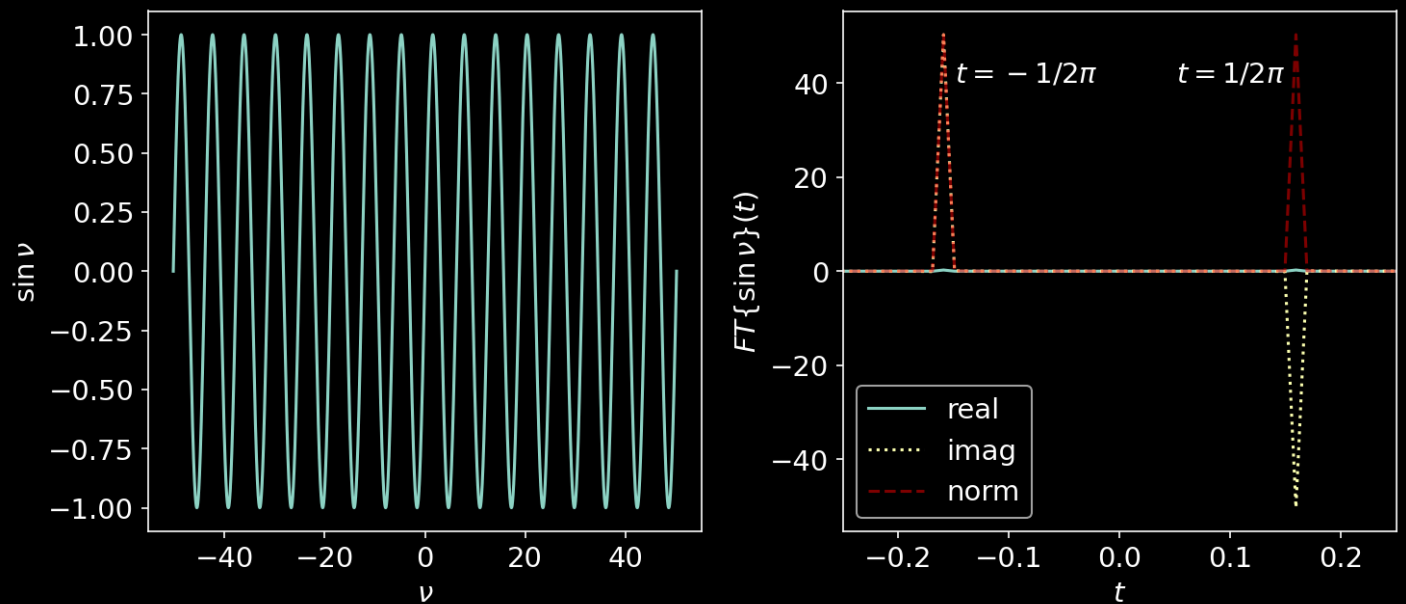
```
In [3]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.cos(nu)
ftx = np.fft.fft(y) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos \nu$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real', linewidth=3)
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color=(1,0.5,0.5),
ax2.text(-1/(2*np.pi) + 0.01, 4000*dnu, r'$t=-1/2\pi$')
ax2.text(1/(2*np.pi) - 0.01, 4000*dnu, r'$t=1/2\pi$', ha='right')
pl.legend(loc='center')
ax2.set_xlim([-0.25, 0.25]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\cos \nu\}(t)$');
pl.suptitle(r'$\cos \nu \leftrightarrow FT\{\cos \nu\}$');
pl.tight_layout()
```

$$\cos \nu \leftrightarrow FT\{\cos \nu\}$$



```
In [4]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.sin(nu)
ftx = np.fft.fft(y) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\sin \nu$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
ax2.text(-1/(2*np.pi) + 0.01, 4000*dnu, r'$t=-1/2\pi$')
ax2.text(1/(2*np.pi) - 0.01, 4000*dnu, r'$t=1/2\pi$', ha='right')
pl.legend(loc='best')
ax2.set_xlim([-0.25, 0.25]); ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\sin \nu\}(t)$');
pl.suptitle(r"$\sin \nu \leftrightarrow FT\{\sin \nu\}$");
pl.tight_layout()
```

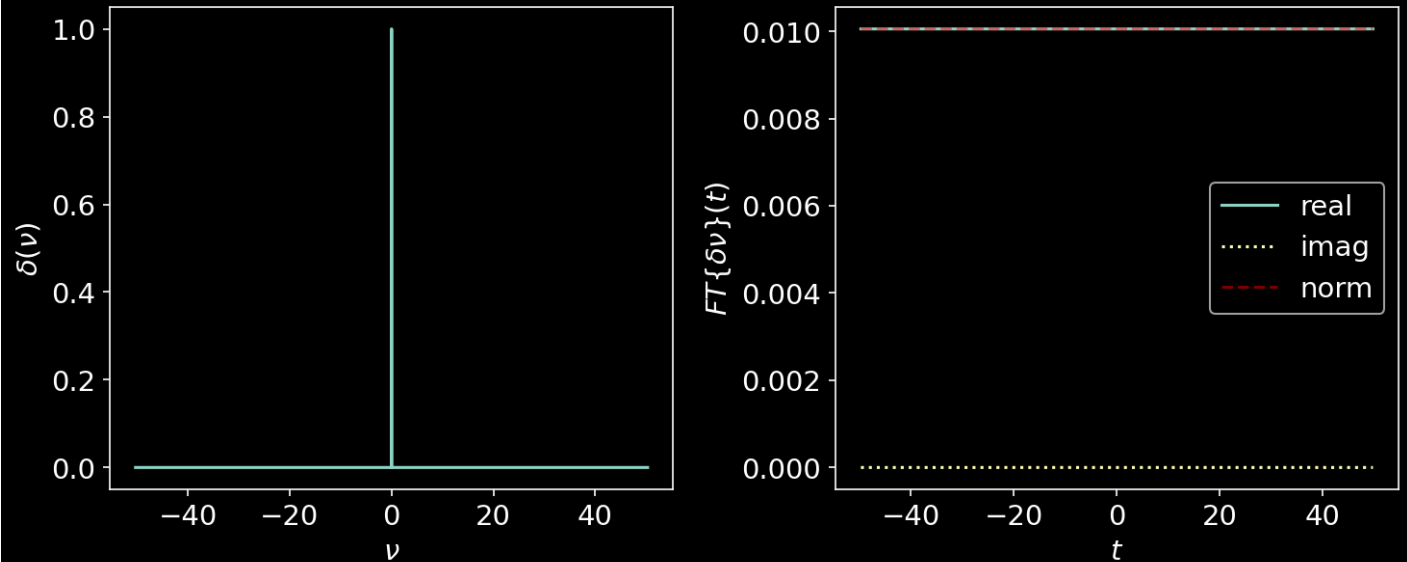
$$\sin \nu \Leftrightarrow FT\{\sin \nu\}$$



```
In [5]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.zeros_like(nu)
y[0] = 1
y = np.fft.fftshift(y)
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta(\nu)\}(t)$');
pl.suptitle(r"$\delta(\nu) \Leftrightarrow FT\{\delta(\nu)\}$"
            "\nDirac Delta");
pl.tight_layout()
```

$$\delta(\nu) \Leftrightarrow FT\{\delta(\nu)\}$$

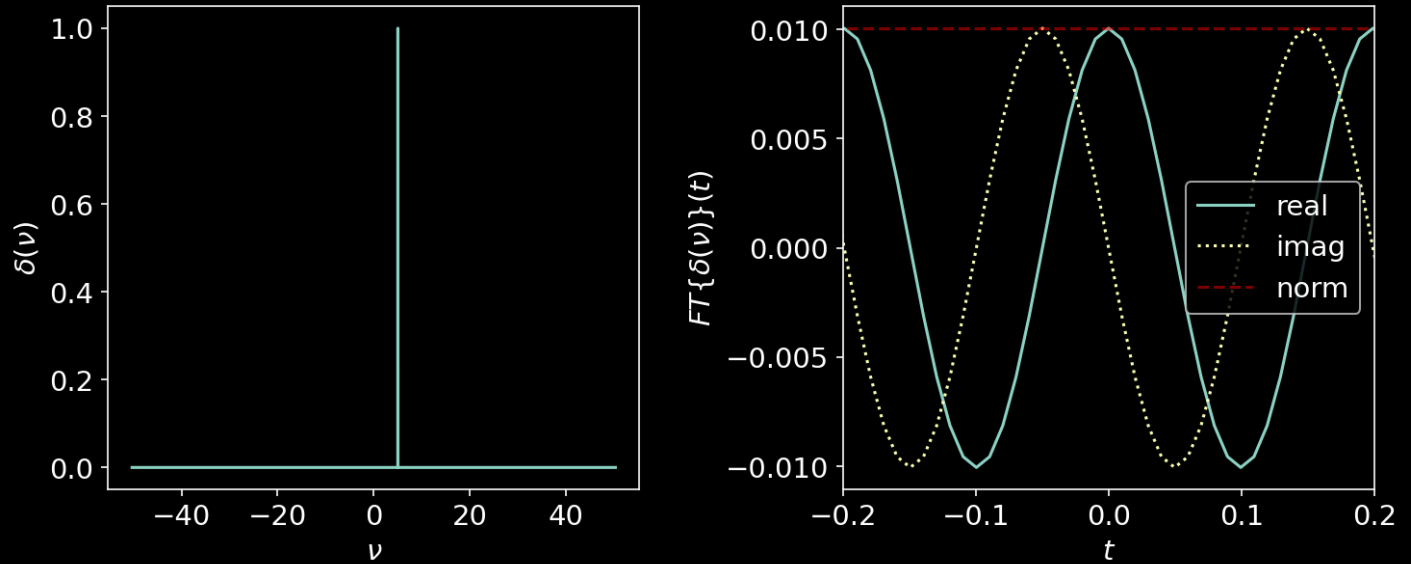
Dirac Delta



```
In [6]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.zeros_like(nu)
y[int(np.floor(nu.size*0.05))] = 1
y = np.fft.fftshift(y)
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
ax2.set_xlim([-0.2, 0.2]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta(\nu)\}(t)$');
pl.suptitle(r"$\delta(\nu-\nu_0) \Leftrightarrow FT\{\delta(\nu-\nu_0)\}$"
            "\n(shift theorem)");
pl.tight_layout()
```

$$\delta(\nu - \nu_0) \leftrightarrow FT\{\delta(\nu - \nu_0)\}$$

(shift theorem)



```
In [7]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1]-nu[0]
y = np.exp(-(nu)**2/(2*np.pi**2))
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax1.plot([0, np.pi], [0.606, 0.606], 'w')
ax1.text(np.pi+1, 0.61, '$\sigma=\pi$')
ax1.set_xlim(-10, 10)
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
ax2.set_xlim([-0.6, 0.6]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta(\nu)\}(t)$');
ax2.plot([0, 1/(2*np.pi**2)], [0.606*y.sum()*dnu, 0.606*y.sum()*dnu], 'w')
ax2.text(1/(2*np.pi**2)+0.01, 500*dnu, '$\sigma_t=1/2 \pi \sigma_{\nu}$')# CORRECT THIS
sigma_t = np.pi
A = 1 * np.sqrt(2*np.pi) * sigma_t
ax2.plot([-0.1, 0], [A, A], linestyle='--', color='w')
ax2.text(-0.55, A*0.93, r'$A_t=A_{\nu} \sigma_{\nu} \sqrt{2 \pi}$')

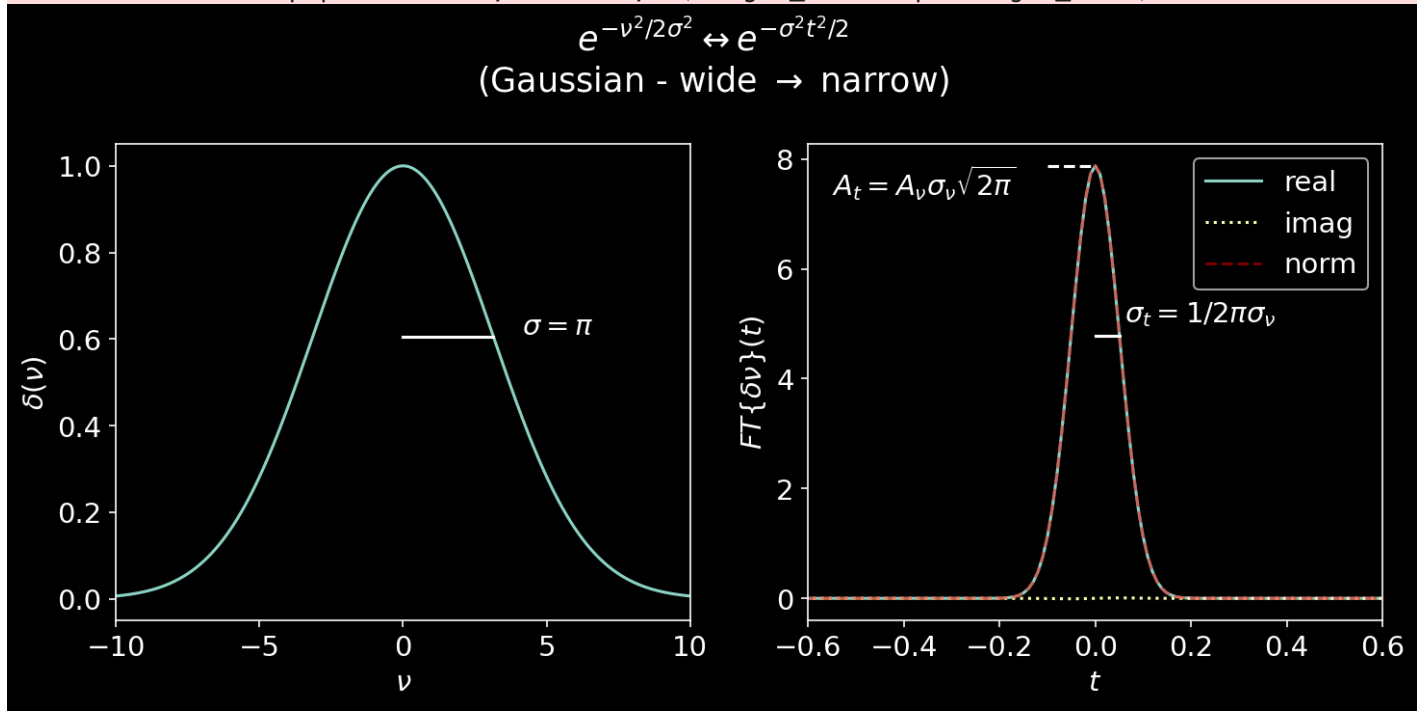
pl.suptitle(r"$e^{-\nu^2/2\sigma^2} \leftrightarrow e^{-\sigma^2 t^2/2}$"
            "\n(Gaussian - wide $\rightarrow$ narrow)");
pl.tight_layout()
```



```

<>:12: SyntaxWarning: invalid escape sequence '\s'
<>:24: SyntaxWarning: invalid escape sequence '\s'
<>:12: SyntaxWarning: invalid escape sequence '\s'
<>:24: SyntaxWarning: invalid escape sequence '\s'
/var/folders/k_/7qh4l0nn72b7qgg15pkd4hw40000gt/T/ipykernel_14794/3998737446.py:12: Syntax
Warning: invalid escape sequence '\s'
  ax1.text(np.pi+1, 0.61, '$\sigma=\pi$')
/var/folders/k_/7qh4l0nn72b7qgg15pkd4hw40000gt/T/ipykernel_14794/3998737446.py:24: Syntax
Warning: invalid escape sequence '\s'
  ax2.text(1/(2*np.pi**2)+0.01, 500*dnu, '$\sigma_t=1/2 \pi \sigma_{\nu}$')# CORRECT THIS

```



PROOF: FT{Gaussian}

$$g(x) = Ae^{-x^2/2\sigma^2}$$

$$FT(f(x)) = G(t) = \int_{-\infty}^{\infty} f(x) \exp[-2\pi ixt] dx$$

$$= \int_{-\infty}^{\infty} Ae^{-x^2/2\sigma^2} \exp[-2\pi ixt] dx$$

$$= \int_{-\infty}^{\infty} A \exp\left[-x^2/2\sigma^2 - 2\pi ixt\right] dx$$

$$= \int_{-\infty}^{\infty} A \exp\left[\frac{-1}{2\sigma^2}(x^2 - 4\pi ixt\sigma^2)\right] dx$$

complete the square

$$= \int_{-\infty}^{\infty} A \exp\left[\frac{-1}{2\sigma^2}\left((x + 2\pi it\sigma^2)^2 + 4\pi^2\sigma^4 t^2\right)\right] dx$$

$$= \exp[-2\pi^2\sigma^2 t^2] \int_{-\infty}^{\infty} A \exp\left[\frac{-1}{2\sigma^2}(x - 2\pi it\sigma^2)^2\right] dx$$

We know that $\int_{-\infty}^{\infty} g(x)dx = A\sigma\sqrt{2\pi}$, independent of translation, so

$$= A\sqrt{2\pi}\sigma \exp[-2\pi^2\sigma^2 t^2]$$

If we define $G(T) = A_t e^{-t^2/2\sigma_t^2}$, we get $A_t = A\sigma_x\sqrt{2\pi}$ and $\sigma_t = \frac{1}{2\pi\sigma_x}$

```
In [8]: nu = np.linspace(-16*np.pi, 16*np.pi, 100001)
dnu = (nu[1]-nu[0])
sigma_t = (0.05*np.pi)
y = np.exp(-(nu)**2/(2*sigma_t**2))
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')

ax1.plot([0, sigma_t], [0.606, 0.606], 'w')
ax1.text(sigma_t+1, 0.61, '$\sigma=0.05\pi$')
ax1.set_xlim(-10, 10)

ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
#ax2.set_xlim([-0.005, 0.005]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta(\nu)(t)\}$');

A = 1 * np.sqrt(2*np.pi) * sigma_t

ax2.plot([0, 1/(2*np.pi * sigma_t)], [0.606*y.sum()*dnu, 0.606*y.sum()*dnu], 'w')
ax2.text(1/(2*np.pi * sigma_t)+0.5, 0.61*y.sum()*dnu, '$\sigma_t=1/2 \pi \sigma_{\nu}$')#

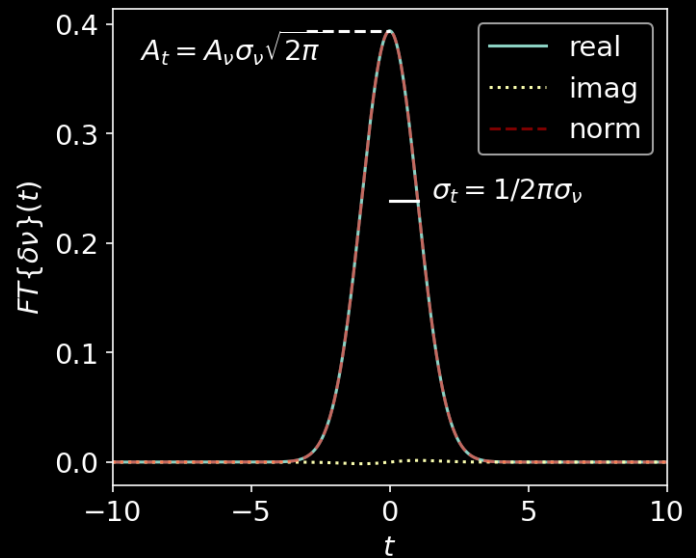
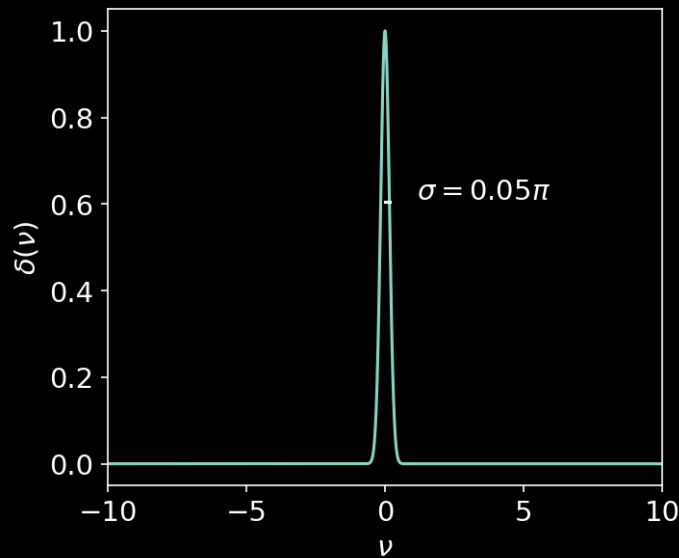
ax2.plot([-3, 0], [A, A], linestyle='--', color='w')
ax2.text(-9, A*0.93, r'$A_t=A_{\nu} \sigma_{\nu} \sqrt{2 \pi}$')
ax2.set_xlim(-10, 10)

pl.suptitle(r"$e^{-\nu^2/2\sigma^2} \rightarrow e^{-\sigma^2 t^2/2}$"
            "\n(Gaussian - narrow$\rightarrow$wide)");
pl.tight_layout()
```

```
<>:14: SyntaxWarning: invalid escape sequence '\s'
<>:30: SyntaxWarning: invalid escape sequence '\s'
<>:14: SyntaxWarning: invalid escape sequence '\s'
<>:30: SyntaxWarning: invalid escape sequence '\s'
/var/folders/k/_7qh4l0nn72b7qqg15pkd4hw40000gt/T/ipykernel_14794/3974604103.py:14: Syntax
Warning: invalid escape sequence '\s'
    ax1.text(sigma_t+1, 0.61, '$\sigma=0.05\pi$')
/var/folders/k/_7qh4l0nn72b7qqg15pkd4hw40000gt/T/ipykernel_14794/3974604103.py:30: Syntax
Warning: invalid escape sequence '\s'
    ax2.text(1/(2*np.pi * sigma_t)+0.5, 0.61*y.sum()*dnu, '$\sigma_t=1/2 \pi \sigma_{\nu}$')
# CORRECT THIS
```

$$e^{-\nu^2/2\sigma^2} \leftrightarrow e^{-\sigma^2 t^2/2}$$

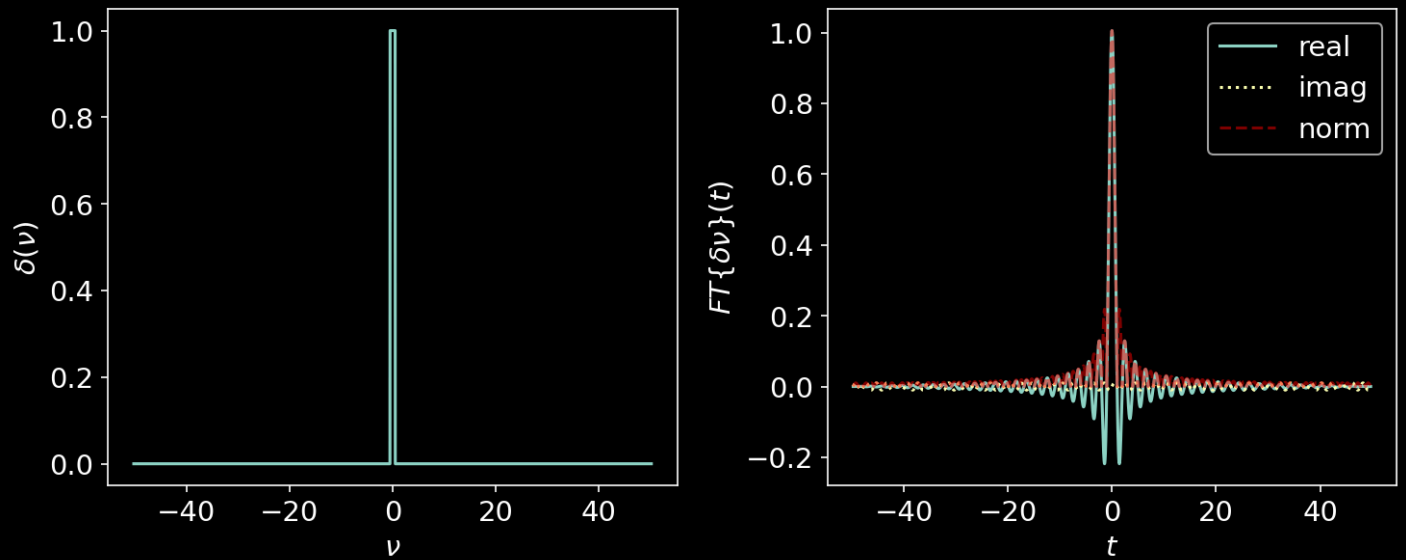
(Gaussian - narrow $\rightarrow$ wide)



```
In [9]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.zeros_like(nu)
y[np.abs(nu)<.5] = 1
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
#ax2.set_xlim([-0.005, 0.005]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta\nu\}(t)$');
pl.suptitle(r"$\text{tophat} \leftrightarrow x^{-1} \sin x$"
            "\n(sinc)");
pl.tight_layout()
```

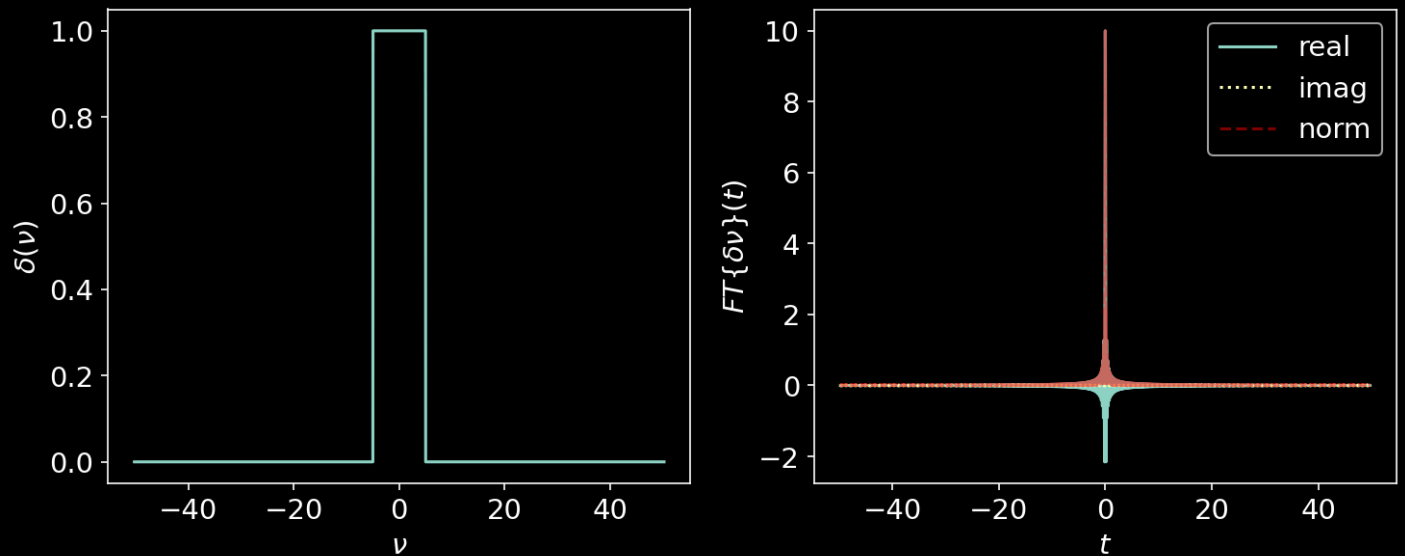
$$\text{tophat} \leftrightarrow x^{-1} \sin x$$

(sinc)



```
In [10]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.zeros_like(nu)
y[np.abs(nu)<5] = 1
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
#ax2.set_xlim([-0.005, 0.005]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta\nu\}(t)$');
pl.suptitle(r"$\text{tophat} \leftrightarrow x^{-1} \sin x$"
            "\n(sinc - wide$\rightarrow$ narrow)");
pl.tight_layout()
```

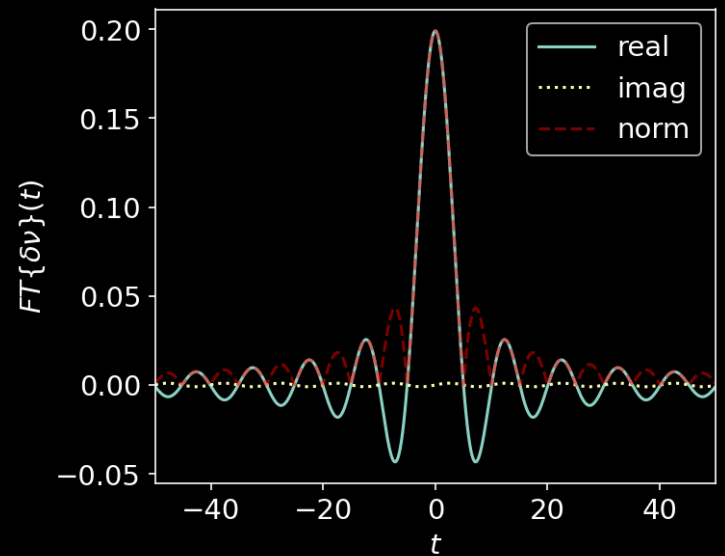
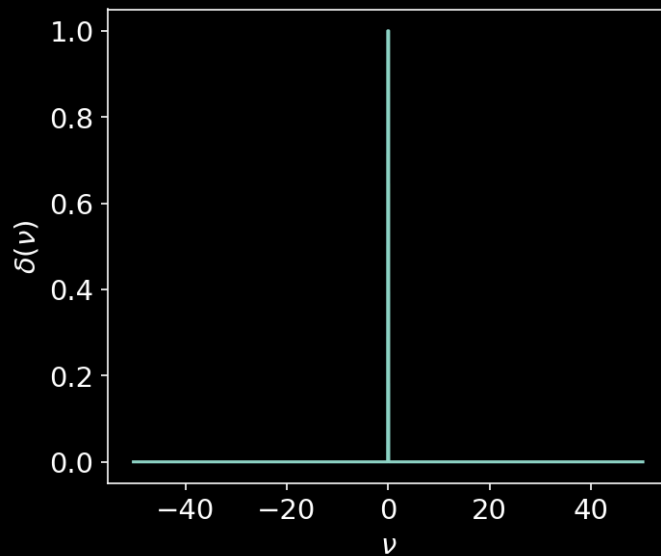
$\text{tophat} \leftrightarrow x^{-1} \sin x$
(sinc - wide $\rightarrow$ narrow)



```
In [11]: nu = np.linspace(-16*np.pi, 16*np.pi, num=100000)
dnu = nu[1]-nu[0]
y = np.zeros_like(nu)
y[np.abs(nu)<0.1] = 1
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
inds = np.argsort(tee)

pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
ax2.set_xlim([-50, 50]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta \nu\}(t)$');
pl.suptitle(r"$\text{tophat} \leftrightarrow x^{-1} \sin x$"
            "\n(sinc - narrow $\rightarrow$ wide)");
pl.tight_layout()
```

$\text{tophat} \leftrightarrow x^{-1} \sin x$
(sinc - narrow $\rightarrow$ wide)

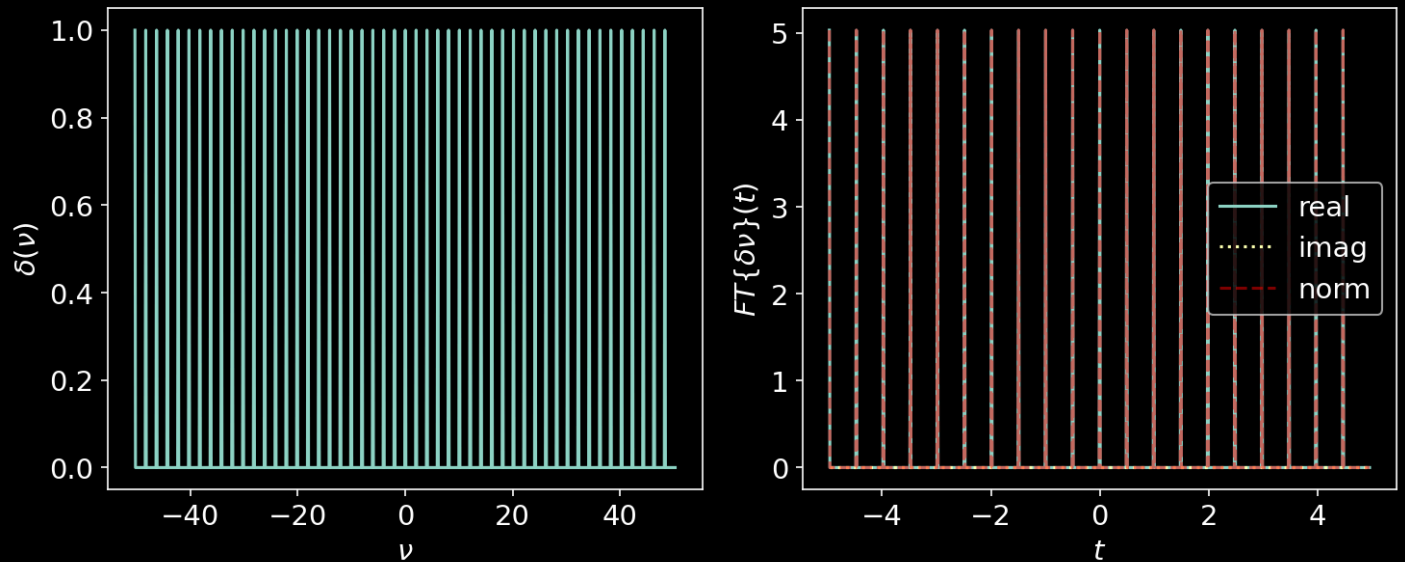


```
In [12]: nu = np.linspace(-16*np.pi, 16*np.pi, 1000)
dnu = nu[1]-nu[0]
y = np.zeros_like(nu)
y[::20] = 1
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
pl.legend(loc='best')
#ax2.set_xlim([-0.005, 0.005]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta \nu\}(t)$');
pl.suptitle(r"$\text{shah} \leftrightarrow \text{shah}$"
            "\n(shah function or Dirac comb - an evenly spaced infinite series of $\delta$)");
pl.tight_layout()
```

```
<>:22: SyntaxWarning: invalid escape sequence '\d'
<>:22: SyntaxWarning: invalid escape sequence '\d'
/var/folders/k/_7qh4l0nn72b7qggq15pkd4hw40000gt/T/ipykernel_14794/986611542.py:22: SyntaxWarning: invalid escape sequence '\d'
      "\n(shah function or Dirac comb - an evenly spaced infinite series of $\delta$)");
```

shah $\leftrightarrow$ *shah*

(shah function or Dirac comb - an evenly spaced infinite series of δ s)



Overview of basic properties

- If a function gets wider (narrower) in the time domain, it gets narrower (wider) in the frequency domain
- If a function is truncated (i.e., zero outside some range in the time domain), it will be infinite in extent in the frequency domain.

DC or Zero-spacing

The zero-frequency component, called the "zero spacing" in interferometry or the Direct Current (DC) in signal processing and electronics, is the integral over the FT domain.

i.e., the value at $f(0)$ is equal to the integral of the Fourier transform:

$$f(0) = \int_{-\infty}^{\infty} g(x)dx = \int_{-\infty}^{\infty} FT\{f(t)\}(x)dx$$

This provides a useful visual check for the amplitude normalization of the function.

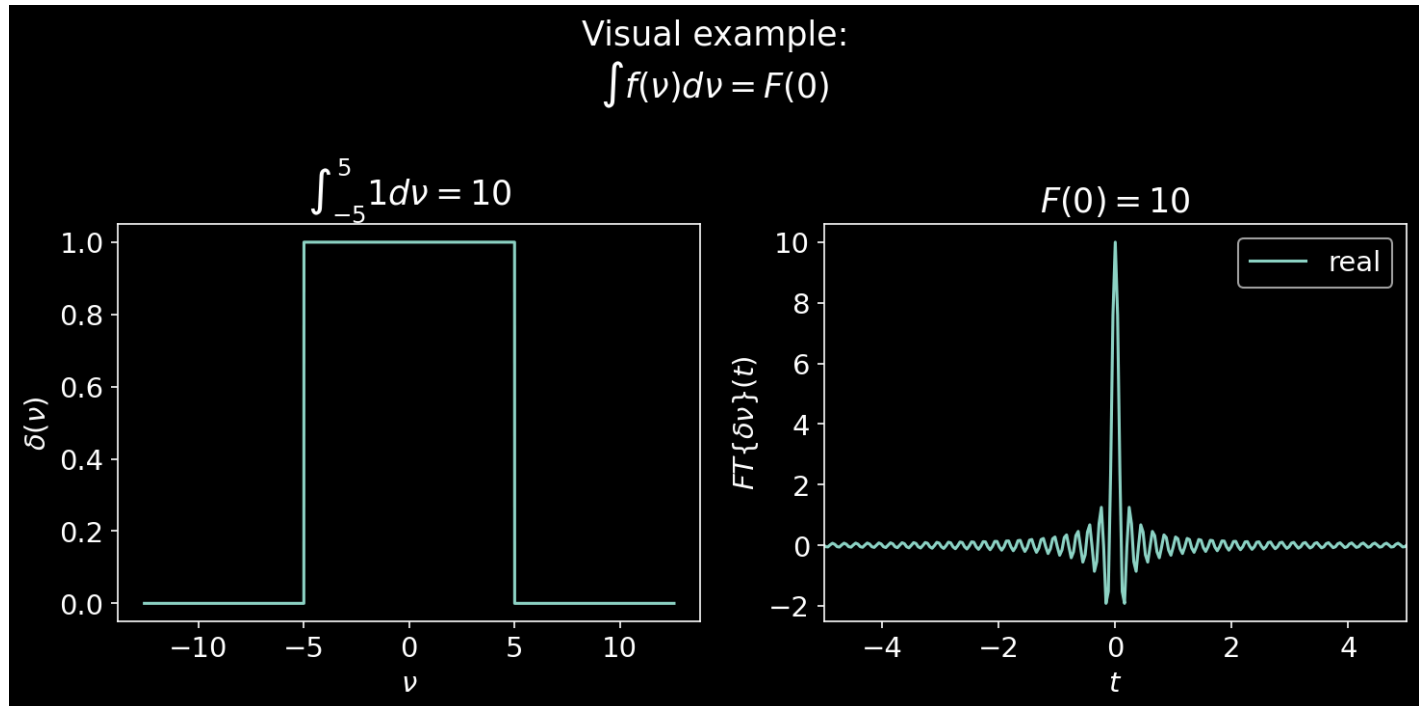
```
In [13]: nu = np.linspace(-4*np.pi, 4*np.pi, num=100000)
dnu = nu[1]-nu[0]
y = np.zeros_like(nu)
y[np.abs(nu)<5] = 1
ftx = np.fft.fft(np.fft.fftshift(y)) * dnu
tee = np.fft.fftfreq(y.size) / dnu
inds = np.argsort(tee)

pl.figure(figsize=(10,5), dpi=150)
ax1 = pl.subplot(1,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\delta(\nu)$')
ax2 = pl.subplot(1,2,2)
ax2.plot(tee[inds], (ftx[inds]).real, label='real')
```

```

#ax2.plot(tee[inds], (ftx[inds]).imag, linestyle=':', label='imag')
#ax2.plot(tee[inds], np.abs(ftx[inds]), linestyle='--', label='norm', color='r', alpha=0.5)
plt.legend(loc='best')
ax2.set_xlim([-5, 5]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\delta\nu\}(t)$');
ax1.set_title(r"$\int_{-5}^5 1 d\nu = 10$")
ax2.set_title("$F(0)=10$")
plt.suptitle("Visual example:\n"
             r"$\int f(\nu) d\nu = F(0)$");
plt.tight_layout()

```



Definition: Fourier Transform Pair

We define the pair $h(t)$ and $H(\nu)$ such that

$$h(t) = FT\{H(\nu)\}$$

i.e., for the next several slides, capital letter functions are the Fourier transforms of lowercase letter functions.

Parseval's Theorem

The total integrated power (i.e., the integral of the norm squared) in the original domain and the Fourier domain are equal. This equality is known as Parseval's theorem.

For a Fourier transform pair $h(t) = FT\{H(\nu)\}$,

$$\int_{-\infty}^{\infty} dt |h(t)|^2 = \int_{-\infty}^{\infty} d\nu |H(\nu)|^2$$

Linearity

Linearity, or superposition theorem: **The FT is a linear operator.**

That is, if

$$h(t) = f(t) + g(t)$$

then

$$H(\nu) = F(\nu) + G(\nu)$$

Even and Odd

Relationships between even/odd functions and real/imaginary functions in the Fourier conjugate domain:

- If $h(t)$ is even, then $H(\nu)$ is real.
 - If $h(t)$ is odd, then $H(\nu)$ is imaginary.
 - If $h(t)$ is real (like sky intensity), then $Re(H(\nu))$ is even, and $Im(H(\nu))$ is odd.
 - If $h(t)$ is imaginary, then $Re(H(\nu))$ is odd, and $Im(H(\nu))$ is even.
-
- If $h(t)$ is even, then $H(\nu)$ is real.
 - If $h(t)$ is odd, then $H(\nu)$ is imaginary.
 - If $h(t)$ is real (like sky intensity), then $Re(H(\nu))$ is even, and $Im(H(\nu))$ is odd.
 - If $h(t)$ is imaginary, then $Re(H(\nu))$ is odd, and $Im(H(\nu))$ is even.

To remember these last relationships, recall that to obtain $H(\nu)$, before integrating, you multiply $h(t)$ by the forward FT kernel,

$$e^{-i2\pi\nu t} = \cos(2\pi\nu t) + i \sin(2\pi\nu t)$$

where $\cos(2\pi\nu t)$ is a real, even function, and $i\sin(2\pi\nu t)$ is an imaginary, odd function.

```
In [14]: nu = np.linspace(-4*np.pi, 4*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.cos(nu)
z = np.exp(-nu**2/(2*(np.pi/2)**2))

pl.figure(figsize=(9,3))
ax1 = pl.subplot(1,3,1)
ax1.axvline(0, linestyle=':')
ax1.axhline(0, linestyle=':')
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos\nu$ ')

ax2 = pl.subplot(1,3,2)
ax2.plot(nu, z)
ax2.set_xlabel(r'$\nu$')
ax2.set_ylabel("Gaussian")

ax3 = pl.subplot(1,3,3)
ax3.plot(nu, z * y)
ax3.set_xlabel(r'$\nu$')
ax3.set_ylabel(r'$\cos\nu \cdot$ Gaussian ')
pl.suptitle("cosine times even function $\rightarrow$ positive integral")
pl.tight_layout()
```

```

nu = np.linspace(-4*np.pi, 4*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.sin(nu)

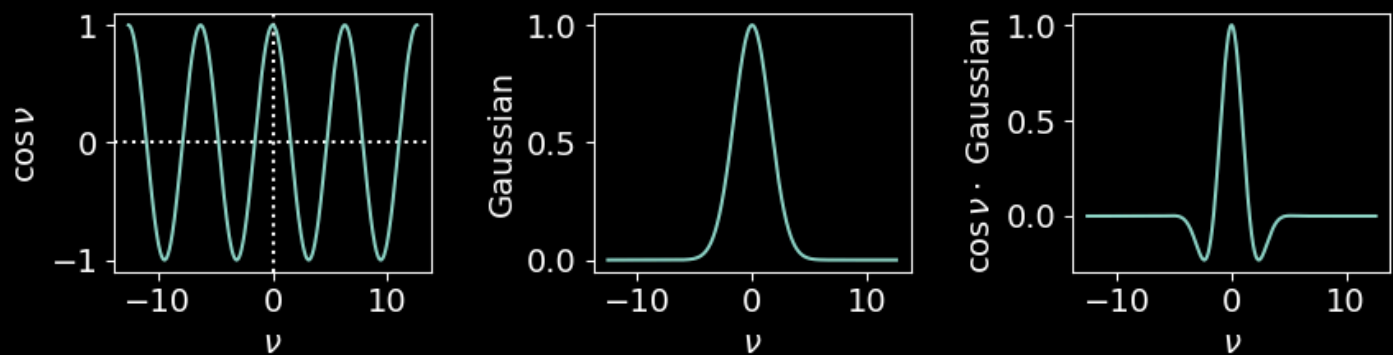
pl.figure(figsize=(9,3))
ax1 = pl.subplot(1,3,1)
ax1.axvline(0, linestyle=':')
ax1.axhline(0, linestyle=':')
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\sin \nu$ ')

ax2 = pl.subplot(1,3,2)
ax2.plot(nu, z)
ax2.set_xlabel(r'$\nu$')
ax2.set_ylabel("Gaussian")

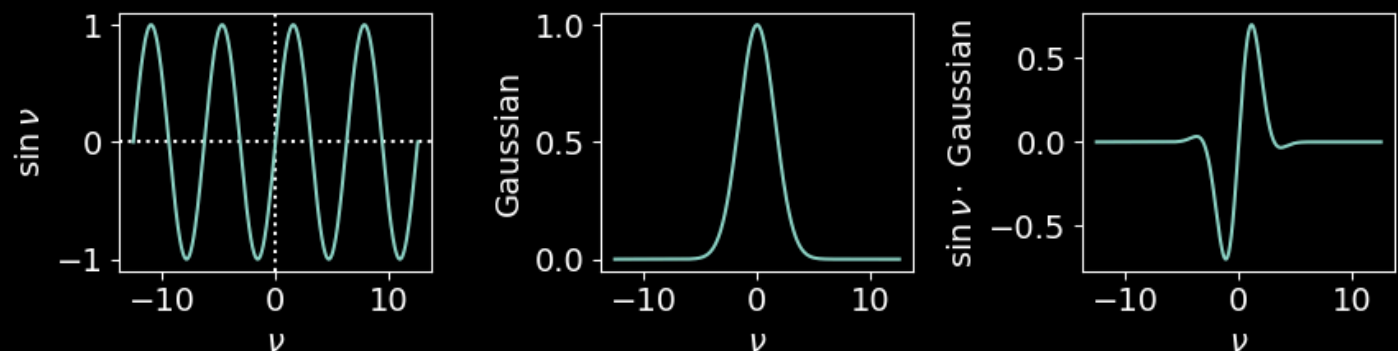
ax3 = pl.subplot(1,3,3)
ax3.plot(nu, z * y)
ax3.set_xlabel(r'$\nu$')
ax3.set_ylabel(r'$\sin \nu \cdot \text{Gaussian}$ ')
pl.suptitle("sine times even function  $\rightarrow$  zero integral")
pl.tight_layout()

```

cosine times even function $\rightarrow$ positive integral



sine times even function $\rightarrow$ zero integral



```

In [15]: nu = np.linspace(-4*np.pi, 4*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.cos(nu)
z = nu/nu.max()

pl.figure(figsize=(9,3))
ax1 = pl.subplot(1,3,1)
ax1.axvline(0, linestyle=':')

```

```

ax1.axhline(0, linestyle=':')
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos \nu$ ')

ax2 = pl.subplot(1,3,2)
ax2.plot(nu, z)
ax2.axvline(0, linestyle=':')
ax2.axhline(0, linestyle=':')

ax2.set_xlabel(r'$\nu$')
ax2.set_ylabel("Linear")

ax3 = pl.subplot(1,3,3)
ax3.plot(nu, z * y)
ax3.axvline(0, linestyle=':')
ax3.axhline(0, linestyle=':')

ax3.set_xlabel(r'$\nu$')
ax3.set_ylabel(r'$\cos \nu \cdot$ Gaussian ')
pl.suptitle("cosine times odd function $\rightarrow$ zero integral")
pl.tight_layout()

nu = np.linspace(-4*np.pi, 4*np.pi, 10000)
dnu = nu[1] - nu[0]
y = np.sin(nu)

pl.figure(figsize=(9,3))
ax1 = pl.subplot(1,3,1)
ax1.axvline(0, linestyle=':')
ax1.axhline(0, linestyle=':')
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\sin \nu$ ')

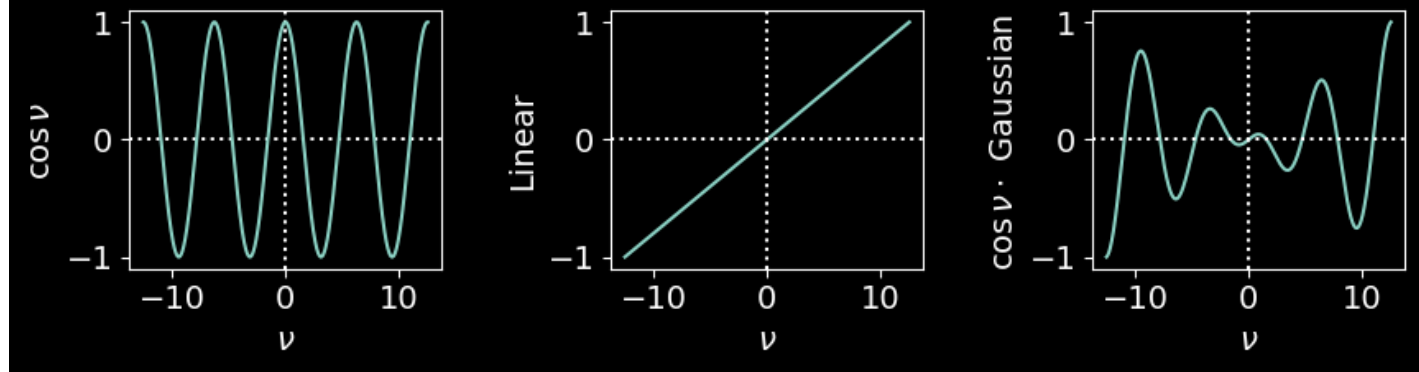
ax2 = pl.subplot(1,3,2)
ax2.axvline(0, linestyle=':')
ax2.axhline(0, linestyle=':')

ax2.plot(nu, z)
ax2.set_xlabel(r'$\nu$')
ax2.set_ylabel("Linear")

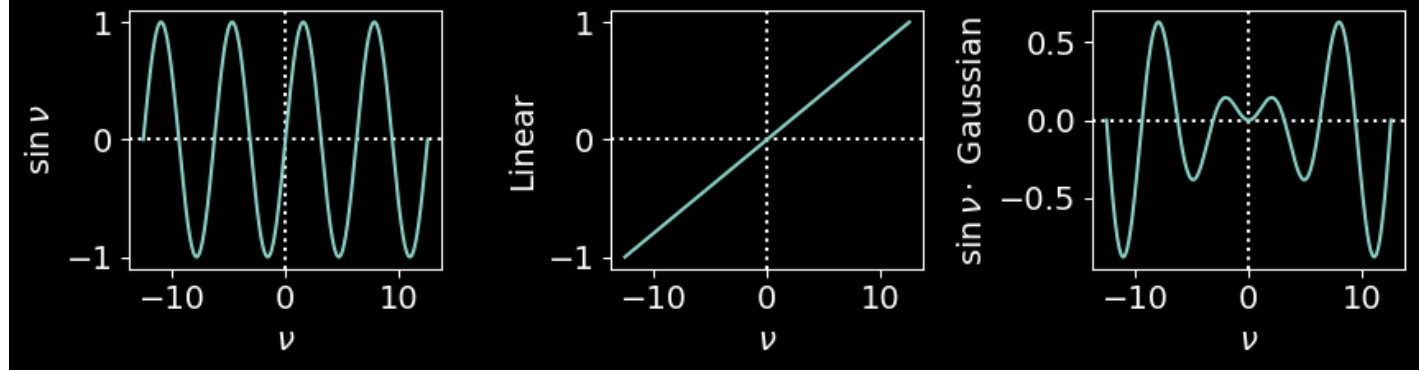
ax3 = pl.subplot(1,3,3)
ax3.axvline(0, linestyle=':')
ax3.axhline(0, linestyle=':')
ax3.plot(nu, z * y)
ax3.set_xlabel(r'$\nu$')
ax3.set_ylabel(r'$\sin \nu \cdot$ Gaussian ')
pl.suptitle("sine times odd function $\rightarrow$ positive integral")
pl.tight_layout()

```

cosine times odd function → zero integral



sine times odd function → positive integral



Convolution

The convolution of two functions $g(t)$, $h(t)$ is given by

$$y(t) = g(t) * h(t) \equiv \int_{-\infty}^{\infty} dt' g(t') h(t - t')$$

Conceptually, to calculate the convolution at some time t , we fold (flip) the function h in the time axis about the y-axis, offset it by amount t , and take the overlap integral, or area, of functions g and the flipped $h(-t)$.

Visual demonstration of convolution:

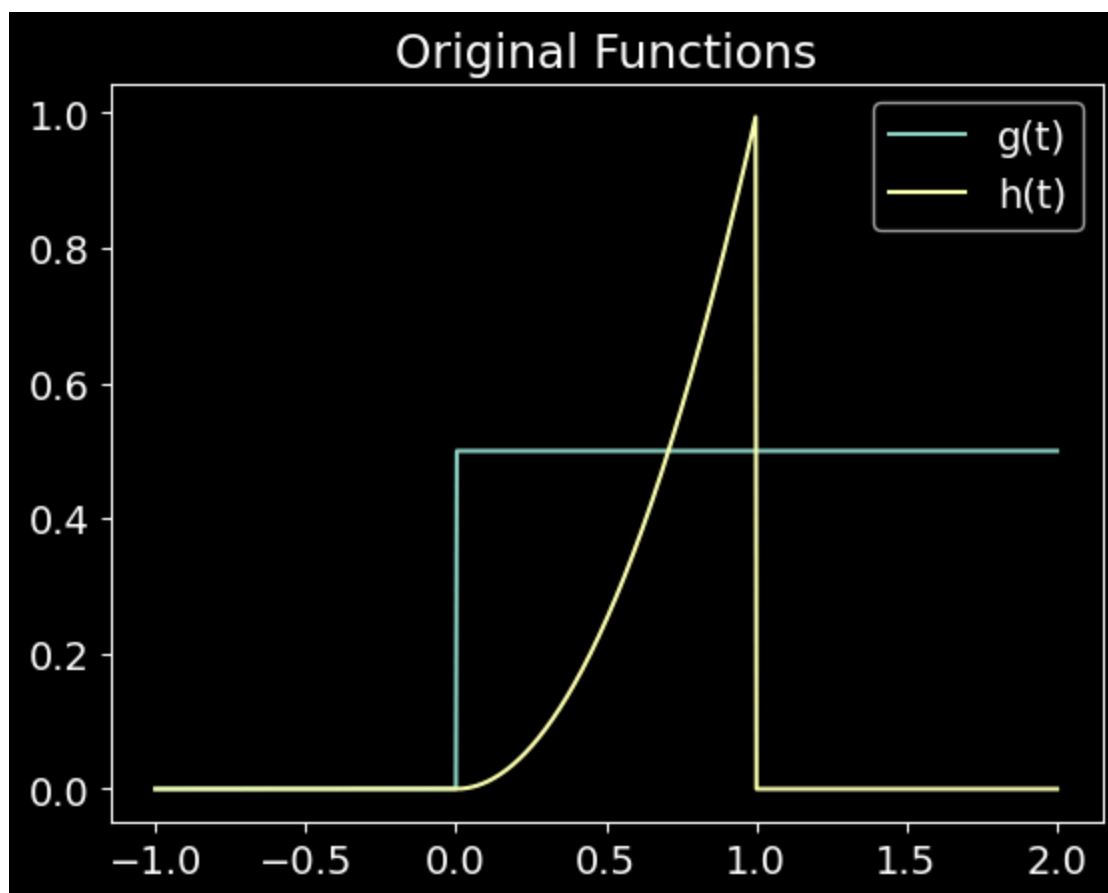
```
In [16]: def g(t):
          return (t > 0)*0.5
          def h(t):
              return t**2 * (t>0) * (t < 1)
```

```
In [17]: def hconvt(t):
          return 0.5 * (1/3 * (t)**3 * ((t>0) & (t<1)) + (t>=1) / 3)
```

```
In [55]: t = np.linspace(-1, 2, 1000)
          pl.plot(t, g(t), label='g(t)')
          pl.plot(t, h(t), label='h(t)')
          pl.legend(loc='best');
          pl.title("Original Functions");
```

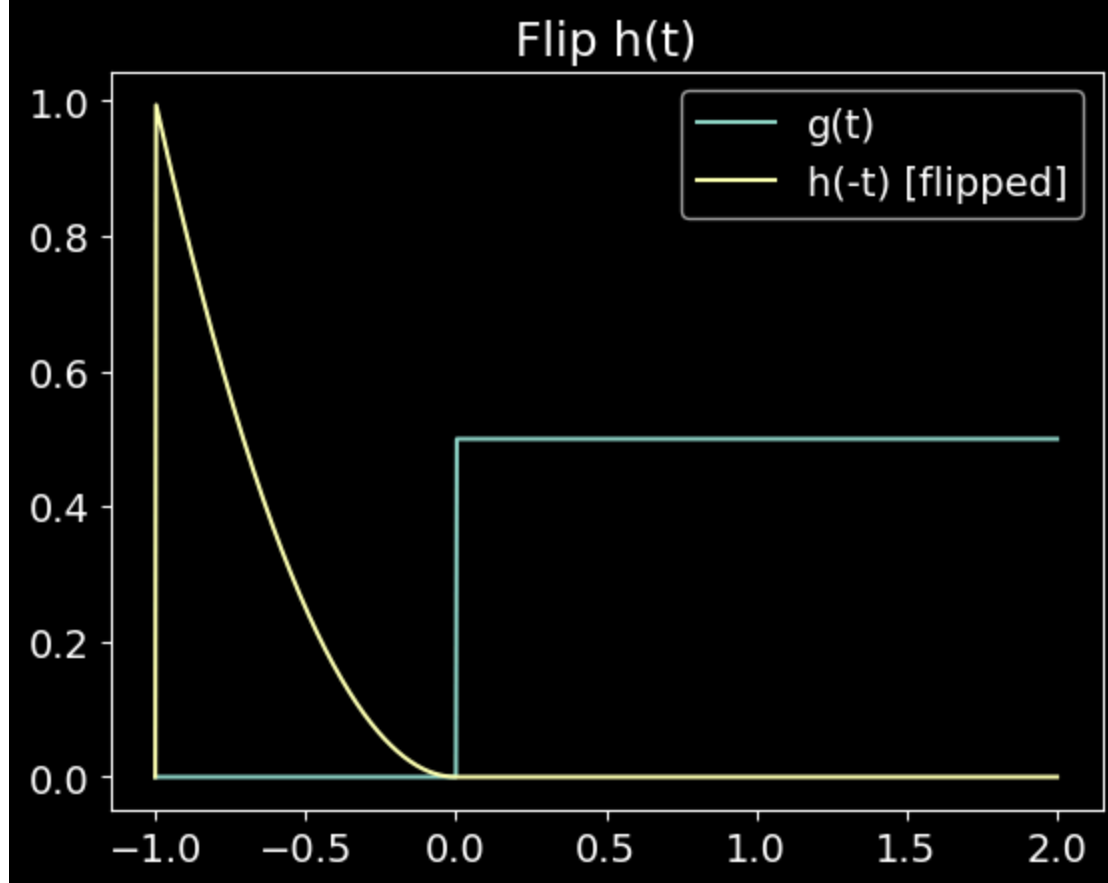
```
from IPython.display import display, Math
Math("$h(t) * g(t)$")
```

Out [55]: $h(t) * g(t)$



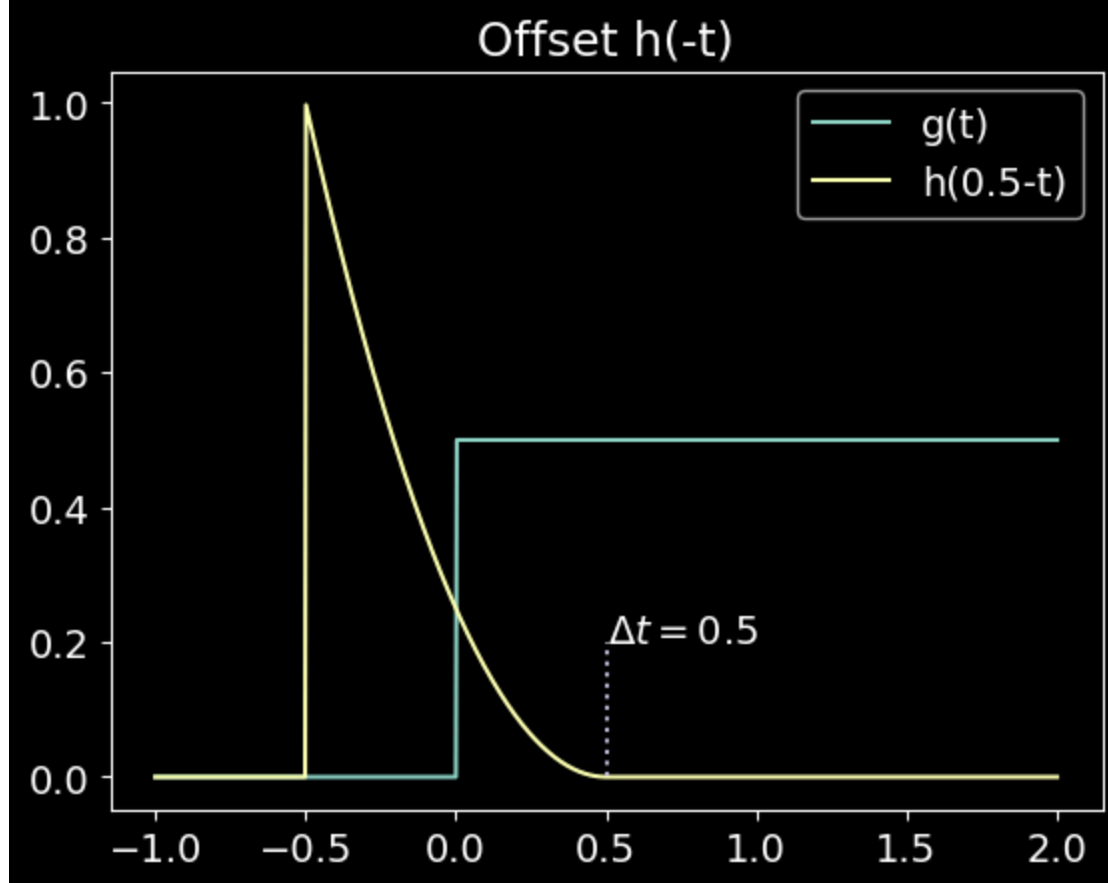
```
In [56]: t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(-t), label='h(-t) [flipped]')
pl.legend(loc='best');
pl.title("Flip h(t)");
Math(r"$g(t)*h(t)\equiv\int_{-\infty}^{\infty} dt' g(t')\color{red}{h(t-t')}$")
```

Out [56]: $g(t) * h(t) \equiv \int_{-\infty}^{\infty} dt' g(t') \color{red}{h(t-t')}$



```
In [57]: t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.5-t), label='h(0.5-t)')
pl.plot([0.5, 0.5,], [0, 0.2], linestyle=':')
pl.text(0.5, 0.2, "$\\Delta t=0.5$")
pl.legend(loc='best');
pl.title("Offset h(-t)");
Math(r"$g(t)*h(t)\equiv\int_{-\infty}^{\infty} dt' g(t')\color{red}{h(t-t')}$")
```

Out [57]:
$$g(t) * h(t) \equiv \int_{-\infty}^{\infty} dt' g(t') \color{red}{h(t-t')}$$

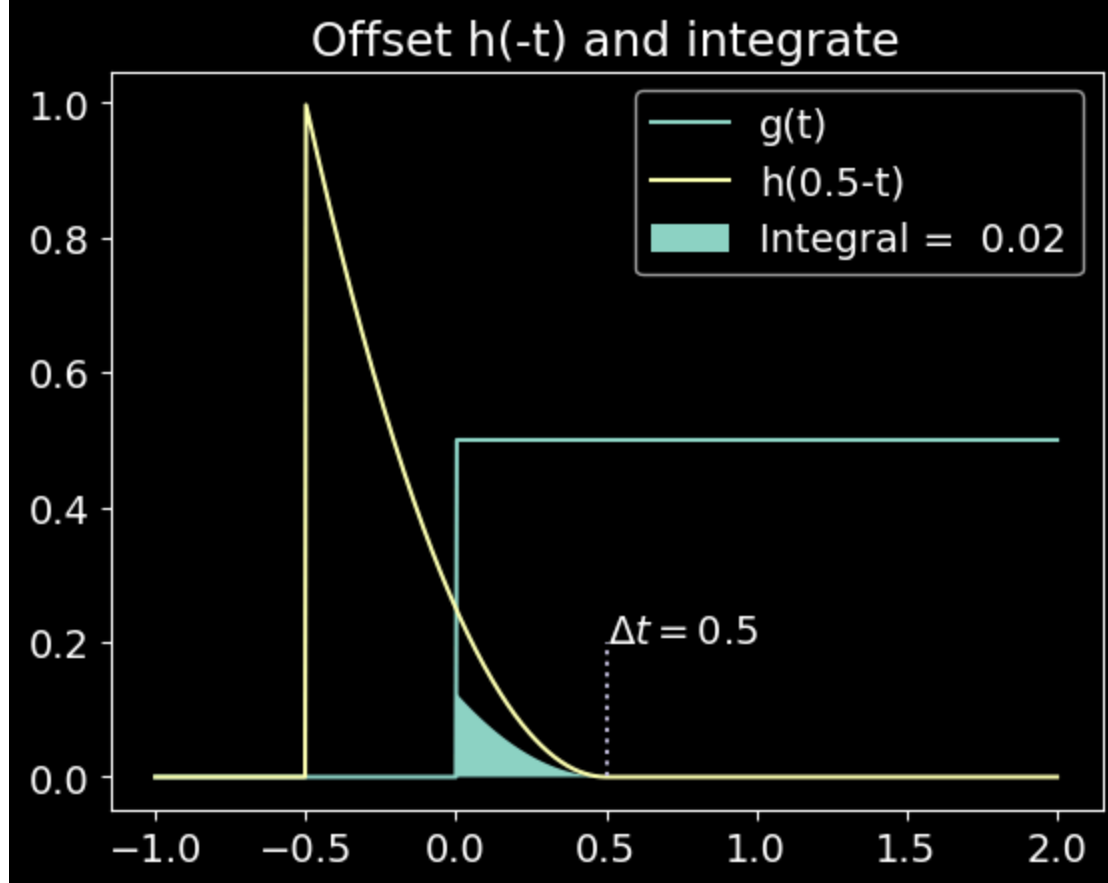


```
In [62]: def overlap_hg(t, dt):
          tmdt = dt - t
          hflip = (tmdt)**2 * (tmdt>0) * (tmdt < 1)
          return hflip * g(t)
```

```
In [63]: t = np.linspace(-1, 2, 1000)
          pl.plot(t, g(t), label='g(t)')
          pl.plot(t, h(0.5-t), label='h(0.5-t)')
          pl.fill_between(t, t*0, overlap_hg(t, 0.5), label=f'Integral = {hconvt(0.5): 0.2f}')
          pl.plot([0.5, 0.5,], [0, 0.2], linestyle=':')
          pl.text(0.5, 0.2, "$\\Delta t=0.5$")

          pl.legend(loc='best');
          pl.title("Offset h(-t) and integrate");
          Math(r"$$$g(t)*h(t)\equiv \color{red}{\int_{-\infty}^{\infty} dt'} g(t') h(t-t')$$$")
```

Out [63]: $g(t) * h(t) \equiv \int_{-\infty}^{\infty} dt' g(t') h(t - t')$

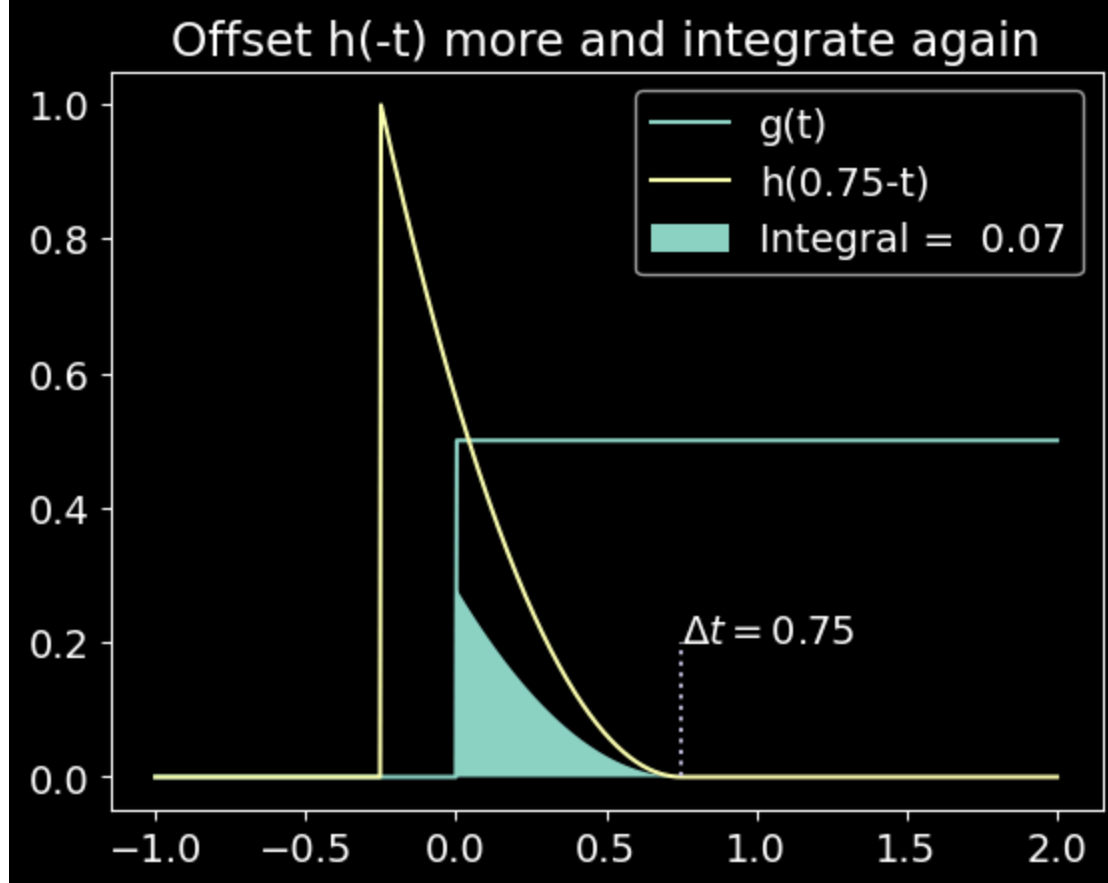


```
In [64]: t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.75-t), label='h(0.75-t)')
pl.fill_between(t, t*0, overlap_hg(t, 0.75), label=f'Integral = {hconvt(0.75): 0.2f}')

pl.plot([0.75, 0.75,], [0, 0.2], linestyle=':')
pl.text(0.75, 0.2, "$\\Delta t=0.75$")

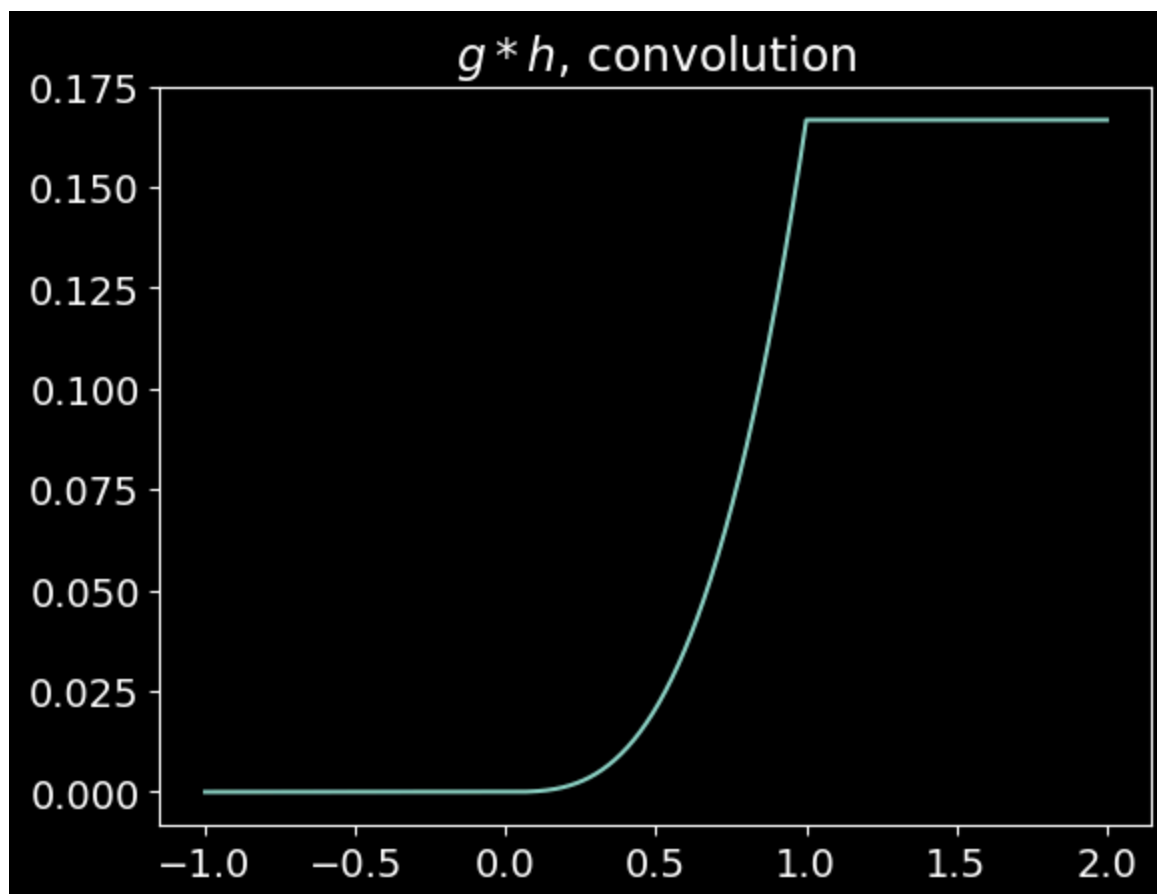
pl.legend(loc='best');
pl.title("Offset h(-t) more and integrate again");
Math(r"$$$g(t)*h(t)\equiv \color{red}{\int_{-\infty}^{\infty} dt'} g(t') h(t-t')$$$")
```

Out [64]: $g(t) * h(t) \equiv \int_{-\infty}^{\infty} dt' g(t') h(t - t')$



```
In [24]: pl.plot(t, hconvt(t));
# manual verification:
#pl.plot(t, [overlap_hg(t, dt).sum() * (t[1]-t[0]) for dt in t], linestyle=':')
pl.title("$g * h$, convolution");
Math("$g(t)*h(t)$")
```

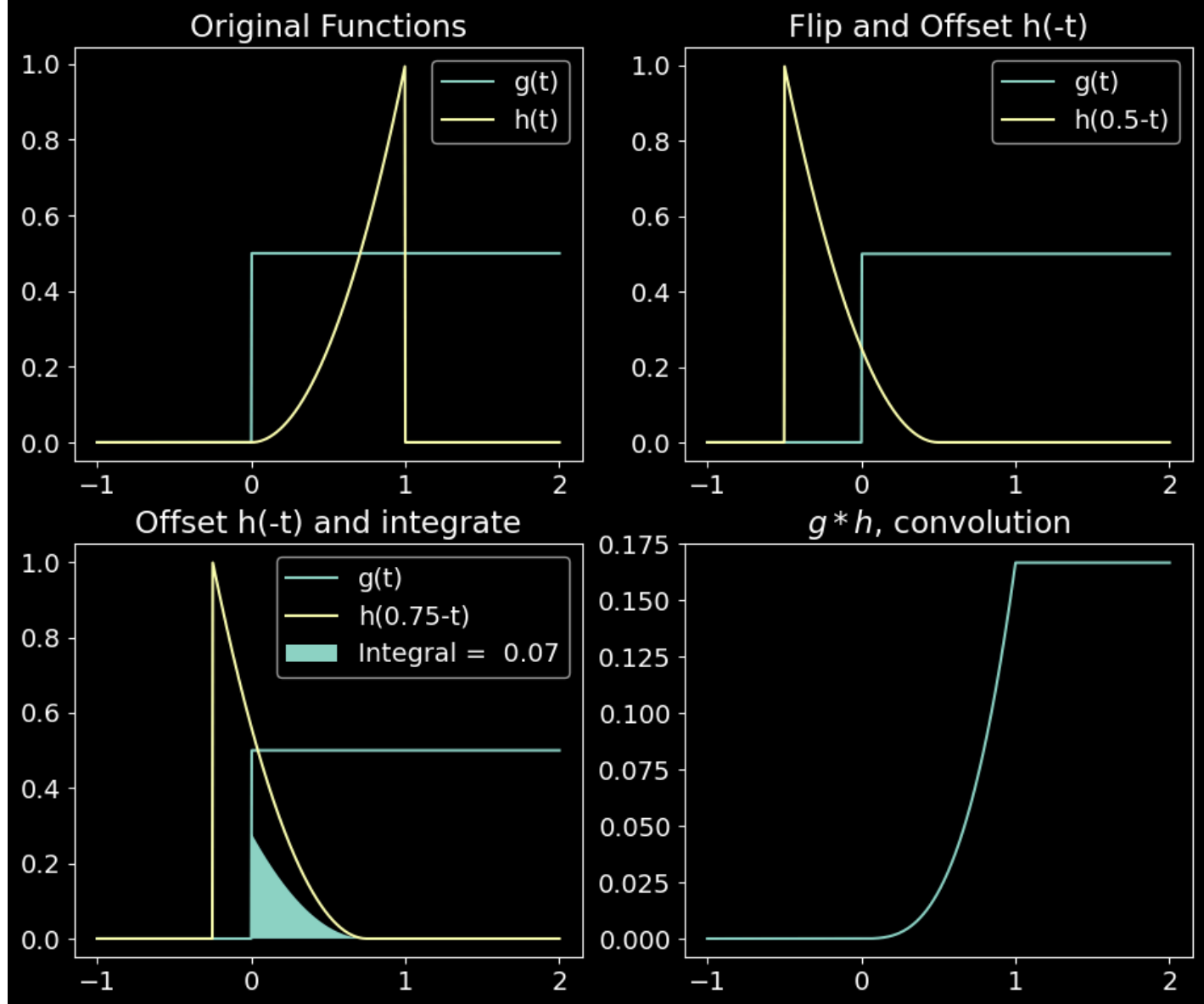
Out[24]: $g(t) * h(t)$



```

In [76]: pl.figure(figsize=(11, 9), dpi=100)
t = np.linspace(-1, 2, 1000)
pl.subplot(2,2,1)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(t), label='h(t)')
pl.legend(loc='best');
pl.title("Original Functions");
pl.subplot(2,2,2)
t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.5-t), label='h(0.5-t)')
pl.legend(loc='best');
pl.title("Flip and Offset h(-t)");
pl.subplot(2,2,3)
t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.75-t), label='h(0.75-t)')
pl.fill_between(t, t*0, overlap_hg(t, 0.75), label=f'Integral = {hconvt(0.75): 0.2f}')
pl.legend(loc='best');
pl.title("Offset h(-t) and integrate");
pl.subplot(2,2,4)
pl.plot(t, hconvt(t));
#pl.plot([0.75, 0.75, t[0]], [0, hconvt(0.75), hconvt(0.75)], linestyle='--', color='w')
#pl.text(0.8, 0.02, 't=0.75', rotation=-90)
#pl.text(-0.5, 0.08, f'$\\int$ = {hconvt(0.75):0.02f}')
pl.title("$g * h$, convolution");

```



```
In [26]: pl.figure(figsize=(11, 9), dpi=100)
t = np.linspace(-1, 2, 1000)
pl.subplot(2,2,1)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(t), label='h(t)')
pl.legend(loc='best');
pl.title("Original Functions");
pl.subplot(2,2,2)
t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.5-t), label='h(0.5-t)')
pl.fill_between(t, t*0, overlap_hg(t, 0.5), label=f'Integral = {hconvt(0.5): 0.2f}')

pl.legend(loc='best');
pl.title("Offset h(-t) and integrate (x=0.5)");
pl.subplot(2,2,3)
t = np.linspace(-1, 2, 1000)
pl.plot(t, g(t), label='g(t)')
pl.plot(t, h(0.75-t), label='h(0.75-t)')
pl.fill_between(t, t*0, overlap_hg(t, 0.75), label=f'Integral = {hconvt(0.75): 0.2f}')

pl.legend(loc='best');
pl.title("Offset h(-t) and integrate (x=0.75)");
pl.subplot(2,2,4)
pl.plot(t, hconvt(t));
pl.plot([-1,0,0.75,0.75], [hconvt(0.75), hconvt(0.75), hconvt(0.75), 0], linestyle=':',
```

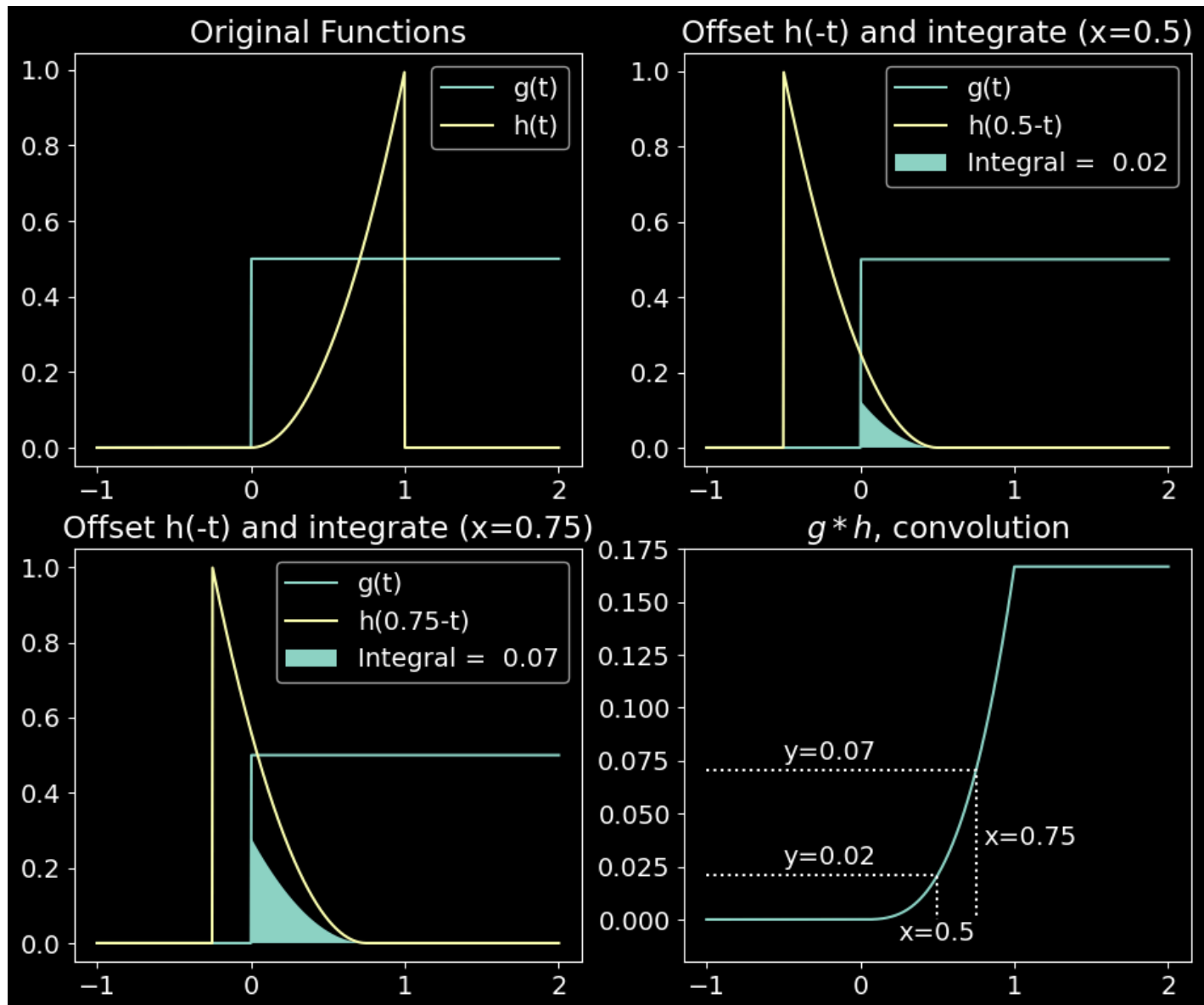
```

pl.text(-0.5, hconvt(0.75)+0.005, f"y={hconvt(0.75):0.2f}")
pl.text(0.8, hconvt(0.75)/2, f"x=0.75")

pl.plot([-1,0,0.5,0.5], [hconvt(0.5), hconvt(0.5), hconvt(0.5), 0], linestyle=':', color
pl.text(-0.5, hconvt(0.5)+0.005, f"y={hconvt(0.5):0.2f}")
pl.text(0.5, -0.01, "x=0.5", ha='center')
ylim = pl.ylim()
pl.ylim(-0.02, ylim[1])

pl.title("$g * h$, convolution");

```



Convolution: Practical Examples

In any linear system, e.g., a measurement device with linear response (so think of a detector where the value it puts out is linearly proportional to the energy put in), the convolution can model the *response* of the system to a given *input*.

In this context, you would think of $g(t)$ as the input and $h(t)$ as the *response* to an infinitesimal impulse (a Dirac Delta input) or the *transfer function*.

Then, $g(t) * h(t)$ is the output of the system.

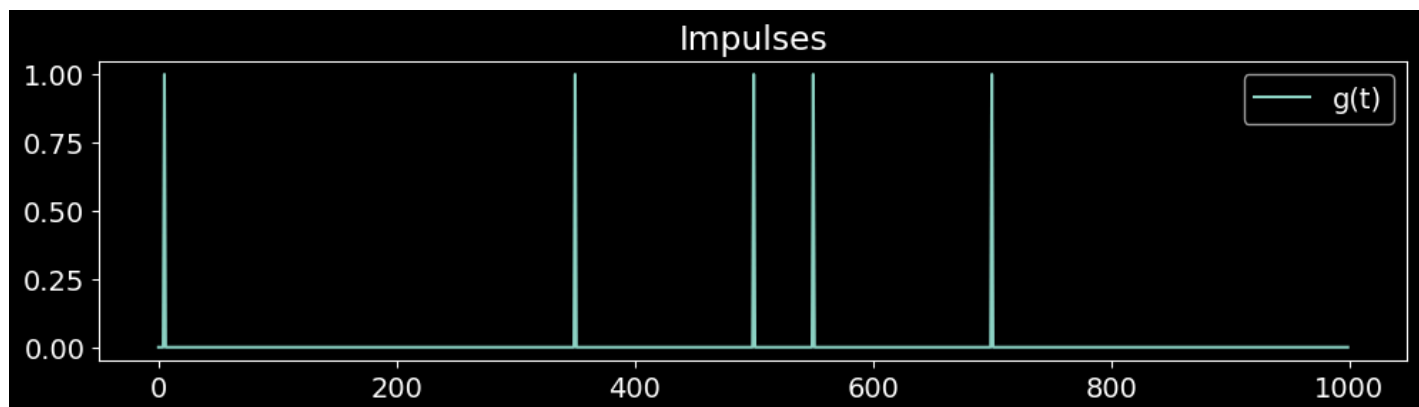
Then, the system output at some time t is the sum of the Delta function response ($h(t)$) over all time lags multiplied by the input g at those time lags.

```
In [27]: impulses = np.array([5, 350, 500, 550, 700])
def g2(t):
    return np.isin(t, impulses)
def h2(t):
    return (((200-t)/200)**2 * (t>0) * (t < 200))
def hconvt2(t):
    return np.fft.ifft(np.fft.fft(g2(t)) *
                       np.fft.fft(h2(t))).real
```

```
In [28]: ts = np.arange(1000)

pl.figure(figsize=(10, 8), dpi=100)
pl.subplot(3,1,1)
pl.plot(ts, g2(ts), label='g(t)')
#pl.axvline(500, linestyle=':')
pl.legend(loc='best');
pl.title("Impulses");

#pl.subplot(3,1,2)
#
#pl.plot(ts, h2(ts), label='h(t)')
#pl.legend(loc='best');
#pl.title("Response Function");
#
##pl.subplot(2,2,3)
##pl.plot(ts, h2(ts[::-1]), label='h(-t)')
##pl.legend(loc='best');
##pl.title("Response Function flipped & shifted");
#
#pl.subplot(3,1,3)
#pl.plot(ts, hconvt2(ts));
#for impulse in impulses:
#    pl.arrow(impulse, impulse*0, impulse*0, impulse*0 + 0.25, zorder=5,
#            length_includes_head=True,
#            head_length=0.05,
#            color='r', head_width=10)
#pl.title("$g * h$, convolution");
pl.tight_layout()
```



```
In [29]: ts = np.arange(1000)

pl.figure(figsize=(10, 8), dpi=100)
```

```

pl.subplot(3,1,1)
pl.plot(ts, g2(ts), label='g(t)')
#pl.axvline(500, linestyle=':')

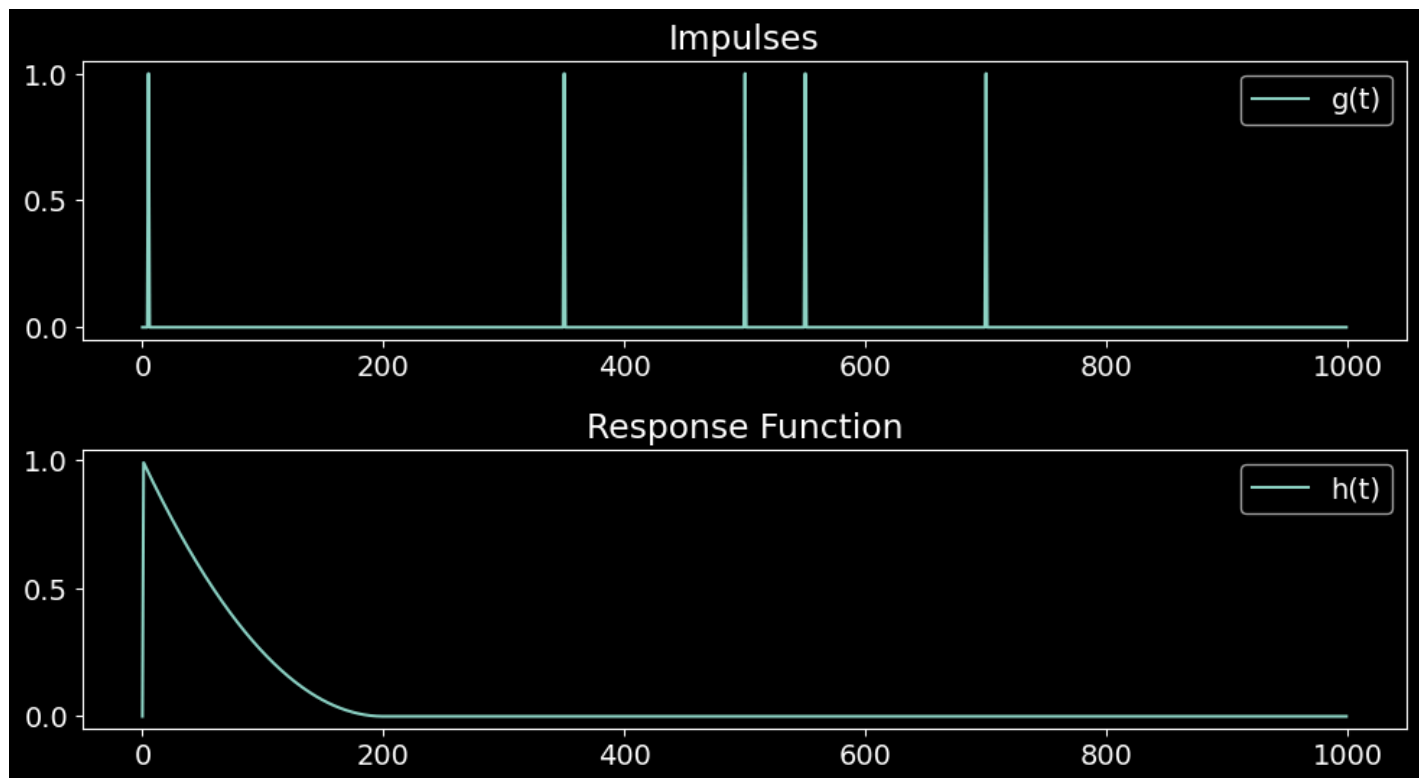
pl.legend(loc='best');
pl.title("Impulses");
pl.subplot(3,1,2)

pl.plot(ts, h2(ts), label='h(t)')
pl.legend(loc='best');
pl.title("Response Function");

#pl.subplot(2,2,3)
#pl.plot(ts, h2(ts[::-1]), label='h(-t)')
#pl.legend(loc='best');
#pl.title("Response Function flipped & shifted");

#pl.subplot(3,1,3)
#pl.plot(ts, hconvt2(ts));
#for impulse in impulses:
#    pl.arrow(impulse, impulse*0, impulse*0, impulse*0 + 0.25, zorder=5,
#            length_includes_head=True,
#            head_length=0.05,
#            color='r', head_width=10)
#pl.title("$g * h$, convolution");
pl.tight_layout()

```



```

In [30]: ts = np.arange(1000)

pl.figure(figsize=(10, 8), dpi=100)
pl.subplot(3,1,1)
pl.plot(ts, g2(ts), label='g(t)')
#pl.axvline(500, linestyle=':')

pl.legend(loc='best');
pl.title("Impulses");
pl.subplot(3,1,2)

```

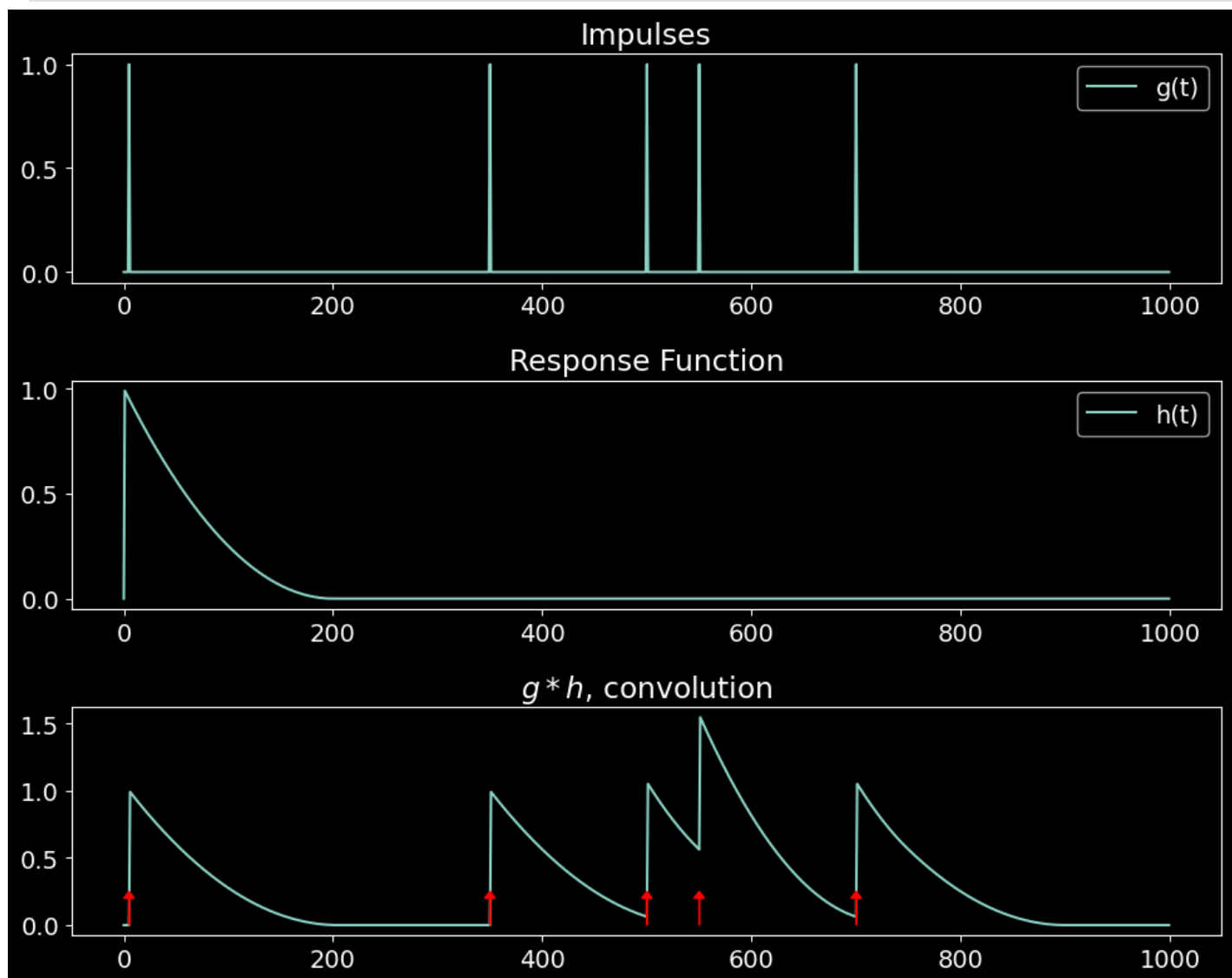
```

pl.plot(ts, h2(ts), label='h(t)')
pl.legend(loc='best');
pl.title("Response Function");

#pl.subplot(2,2,3)
#pl.plot(ts, h2(ts[::-1]), label='h(-t)')
#pl.legend(loc='best');
#pl.title("Response Function flipped & shifted");

pl.subplot(3,1,3)
pl.plot(ts, hconv2(ts));
for impulse in impulses:
    pl.arrow(impulse, impulse*0, impulse*0, impulse*0 + 0.25, zorder=5,
            length_includes_head=True,
            head_length=0.05,
            color='r', head_width=10)
pl.title("$g * h$, convolution");
pl.tight_layout()

```



Another example of a convolution is a simple optical system.

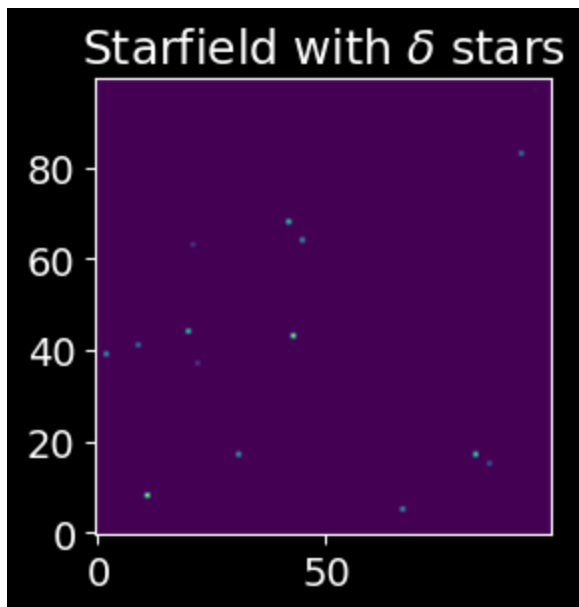
The focal plane image is the *true* sky brightness distribution $I_{sky}(\theta, \phi)$ convolved with the telescope response function, the PSF:

$$I_{\text{focal plane}}(\theta, \phi) = \int \int d\Omega I_{sky}(\theta', \phi') A(\theta - \theta', \phi - \phi')$$

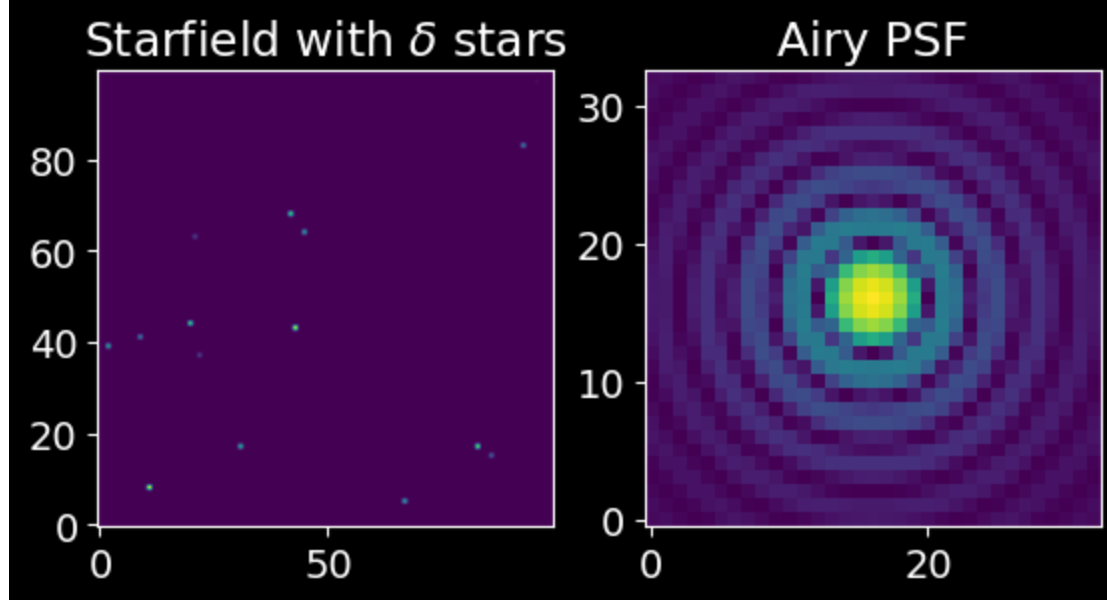
where $A(\theta - \theta', \phi - \phi')$ is the telescope response function (point spread function). In this case, the convolution is 2-dimensional.

```
In [31]: from astropy.convolution import convolve, AiryDisk2DKernel
from astropy.visualization import simple_norm
starfield = np.zeros([100,100])
psf = AiryDisk2DKernel(4)
starfield.flat[np.random.randint(0, starfield.size, 15)] = np.random.rand(15)
```

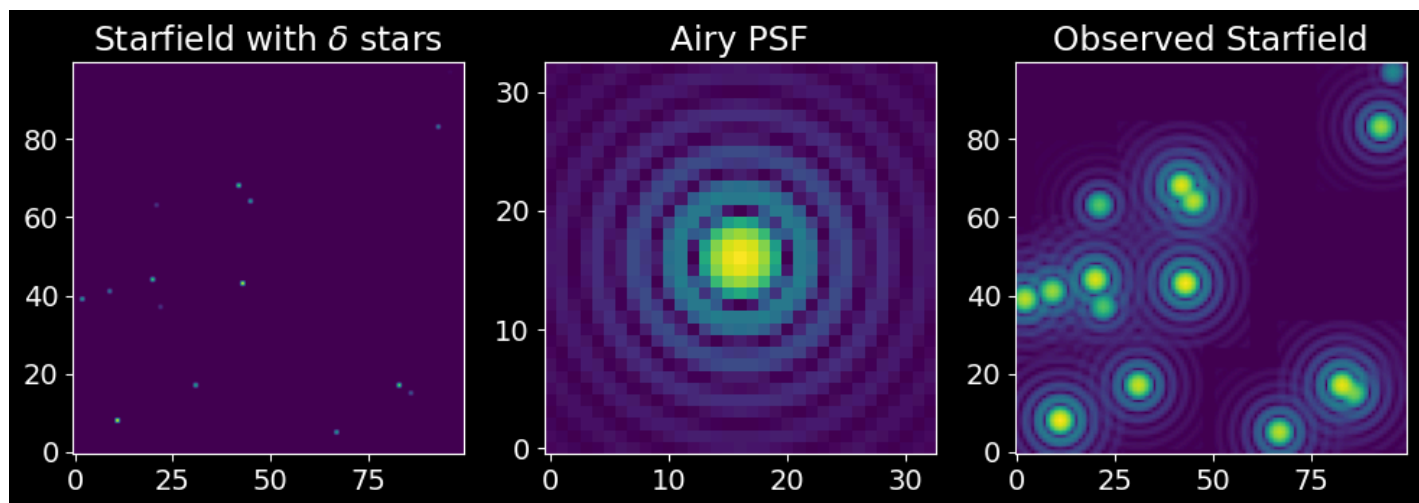
```
In [50]: pl.figure(figsize=(10,5))
pl.subplot(1,3,1)
pl.imshow(starfield)
pl.title("Starfield with  $\delta$  stars");
#pl.subplot(1,3,2)
#pl.imshow(psf.array, norm=simple_norm(psf.array, stretch='log'))
#pl.title("Airy PSF")
#pl.subplot(1,3,3)
#conv_starfield = convolve(starfield, psf)
#pl.imshow(conv_starfield, norm=simple_norm(conv_starfield, stretch='log'))
#pl.title("Observed Starfield")
#pl.tight_layout()
```



```
In [49]: pl.figure(figsize=(10,5))
pl.subplot(1,3,1)
pl.imshow(starfield)
pl.title("Starfield with  $\delta$  stars")
pl.subplot(1,3,2)
pl.imshow(psf.array, norm=simple_norm(psf.array, stretch='log'))
pl.title("Airy PSF");
#pl.subplot(1,3,3)
#conv_starfield = convolve(starfield, psf)
#pl.imshow(conv_starfield, norm=simple_norm(conv_starfield, stretch='log'))
#pl.title("Observed Starfield")
#pl.tight_layout()
```

```
In [48]: pl.figure(figsize=(10,5))
pl.subplot(1,3,1)
pl.imshow(starfield)
pl.title("Starfield with  $\delta$  stars")
pl.subplot(1,3,2)
pl.imshow(psf.array, norm=simple_norm(psf.array, stretch='log'))
pl.title("Airy PSF")
pl.subplot(1,3,3)
conv_starfield = convolve(starfield, psf)
pl.imshow(conv_starfield, norm=simple_norm(conv_starfield, stretch='log'))
pl.title("Observed Starfield")
pl.tight_layout()
```



Convolution Theorem

The convolution in frequency is equal to the Fourier transform of the product in time.

$$FT\{g(t) * h(t)\} = G(f) \cdot H(f)$$

We'll demonstrate and use this.

Notational clarification / reminder:

$$FT\{g(t)\} \equiv G(f)$$

so

$$FT\{g(t) * h(t)\} = FT\{g(t)\} \cdot FT\{h(t)\}$$

Since multiplication is commutative, and the Fourier transform is a linear operation, the convolution theorem also implies that convolution is commutative, i.e.:

$$g(t) * h(t) = h(t) * g(t)$$

Visual Fourier Transforms using the Convolution theorem

```
In [35]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.cos(nu)
ftx = np.fft.fft(y)
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])
fig = plt.figure(figsize=(10,10), dpi=90)
ax1 = plt.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos \nu$')
ax1.set_title("cosine")

ax2 = plt.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.25, 0.25]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\cos \nu\}(t)$');
ax2.set_title(r"Two $\delta s$")

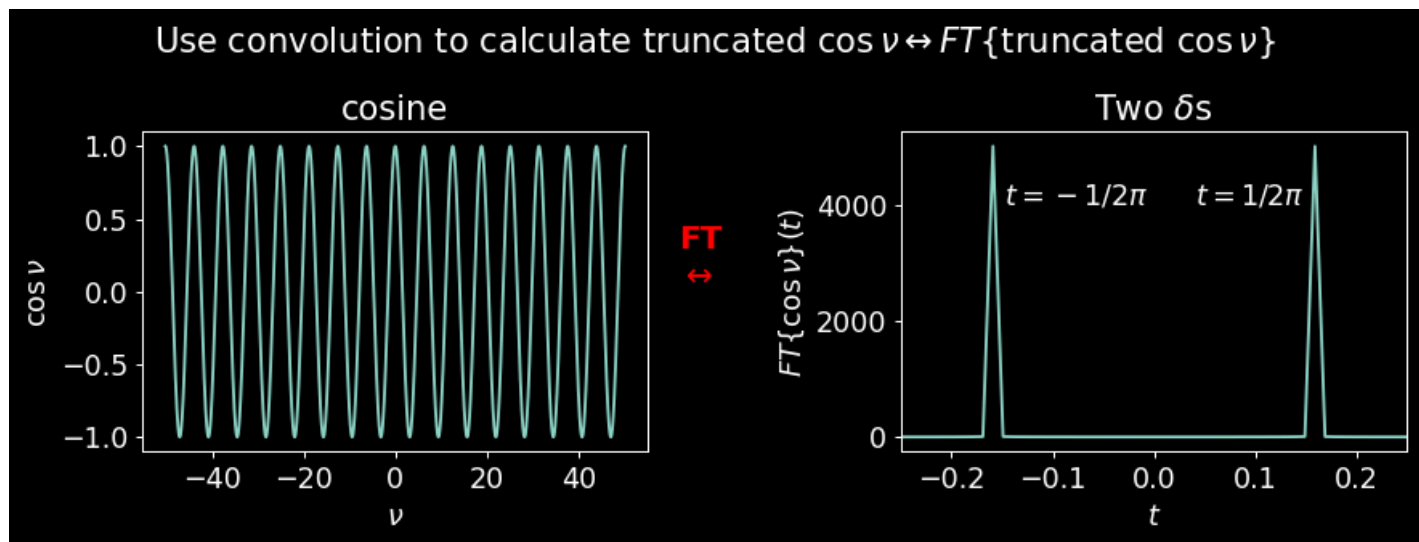
# ax3 = plt.subplot(3,2,3)
# trunc = (nu > -4*np.pi) & (nu < 4*np.pi)
# ax3.plot(nu, trunc)
# ax3.set_xlabel('$\nu$')
# ax3.set_ylabel(r'Boxcar');
#
#
# ax5 = plt.subplot(3,2,5)
# ax5.plot(nu, trunc * y)
# ax5.set_xlabel('$\nu$')
# ax5.set_ylabel(r'$\cos \nu \cdot \text{Boxcar}$');
#
#
# ax4 = plt.subplot(3,2,4)
# inds = np.argsort(tee)
# ftx2 = np.fft.fft(trunc)
# ax4.plot(tee[inds], np.abs(ftx2[inds]), )
# ax4.set_xlim([-0.25, 0.25]);
# ax4.set_xlabel('$t$')
#
# ax4.set_ylabel(r'$FT\{\text{Boxcar}\}(t)$');
#
#
# ax6 = plt.subplot(3,2,6)
```

```

# inds = np.argsort(tee)
# ftx3 = np.fft.fft(trunc*y)
# ax6.plot(tee[inds], np.abs(ftx3[inds]), )
# ax6.set_xlim([-0.4, 0.4]);
# ax6.set_ylabel(r'$FT\{\cos \nu \cdot \text{Boxcar}\}(t)$');
# ax6.set_xlabel('$t$')

pl.suptitle(r"Use convolution to calculate truncated  $\cos \nu \leftrightarrow FT\{\text{truncated } \cos \nu\}$ ")
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5);
pl.text(0.475, 0.8, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.475, 0.5, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.475, 0.1, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.27, 0.62, "$\cdot$ [multiplication]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.8, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
#pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)

```



```

In [47]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.cos(nu)
ftx = np.fft.fft(y)
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])
fig = pl.figure(figsize=(10,10), dpi=90)
ax1 = pl.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos \nu$')

ax2 = pl.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.25, 0.25]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\cos \nu\}(t)$');

ax3 = pl.subplot(3,2,3)
trunc = (nu > -4*np.pi) & (nu < 4*np.pi)
ax3.plot(nu, trunc)
ax3.set_xlabel('$\nu$')
ax3.set_ylabel(r'$\text{Boxcar}$');

```

```

#ax5 = pl.subplot(3,2,5)
#ax5.plot(nu, trunc * y)
#ax5.set_xlabel('$\\nu$')
#ax5.set_ylabel(r'$\cos \nu \cdot \text{Boxcar}$');

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(trunc)
ax4.plot(tee[inds], (ftx2[inds]).real )
ax4.set_xlim([-0.25, 0.25]);
ax4.set_xlabel('$t$')

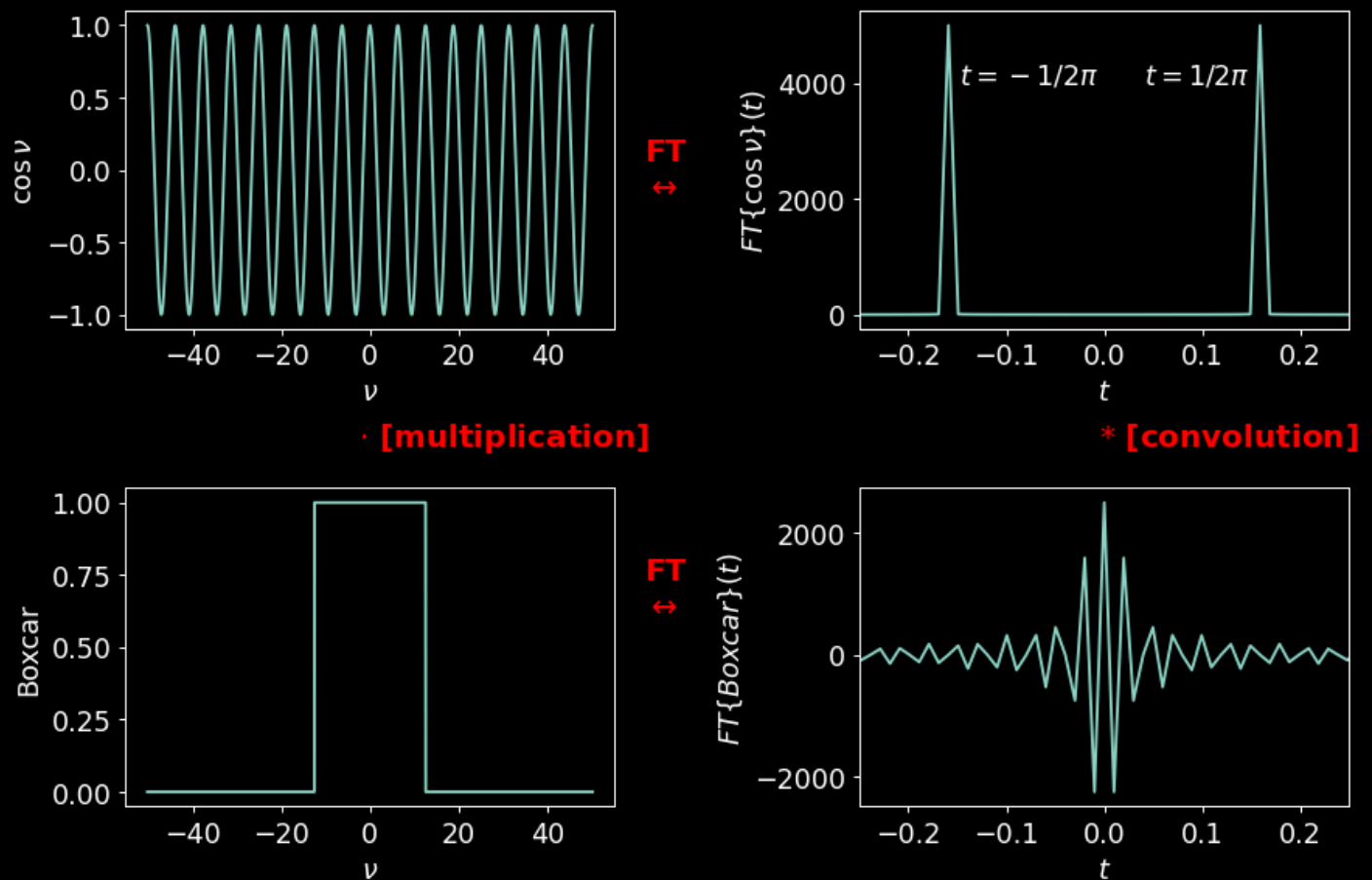
ax4.set_ylabel(r'$\mathcal{FT}\{\text{Boxcar}\}(t)$');

# ax6 = pl.subplot(3,2,6)
# inds = np.argsort(tee)
# ftx3 = np.fft.fft(trunc*y)
# ax6.plot(tee[inds], np.abs(ftx3[inds]), )
# ax6.set_xlim([-0.4, 0.4]);
# ax6.set_ylabel(r'$\mathcal{FT}\{\cos \nu \cdot \text{Boxcar}\}(t)$');
# ax6.set_xlabel('$t$')

pl.suptitle(r"Use convolution to calculate truncated  $\cos \nu \leftrightarrow \mathcal{FT}\{\mathcal{FT}\{$ 
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, " $\mathcal{FT}\{ \leftrightarrow$ ", transform=fig.transFigure, color='r', weig
pl.text(0.475, 0.5, " $\mathcal{FT}\{ \leftrightarrow$ ", transform=fig.transFigure, color='r', weig
#pl.text(0.475, 0.1, " $\mathcal{FT}\{ \leftrightarrow$ ", transform=fig.transFigure, color='r', wei
pl.text(0.27, 0.62, " $\cdot$  [multiplication]", transform=fig.transFigure, color='r', w
pl.text(0.8, 0.62, " $*$  [convolution]", transform=fig.transFigure, color='r', weight='bo
#pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=
#pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=1

```

Use convolution to calculate truncated $\cos v \leftrightarrow FT\{\text{truncated } \cos v\}$



```
In [46]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.cos(nu)
ftx = np.fft.fft(y)
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])
fig = plt.figure(figsize=(10,10), dpi=90)
ax1 = plt.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'$\cos \nu$')

ax2 = plt.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.25, 0.25]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\cos \nu\}(t)$');

ax3 = plt.subplot(3,2,3)
trunc = (nu > -4*np.pi) & (nu < 4*np.pi)
ax3.plot(nu, trunc)
ax3.set_xlabel('$\nu$')
ax3.set_ylabel(r'Boxcar');

ax5 = plt.subplot(3,2,5)
ax5.plot(nu, trunc * y)
ax5.set_xlabel('$\nu$')
ax5.set_ylabel(r'$\cos \nu \cdot \text{Boxcar}$');
```

```

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(trunc)
ax4.plot(tee[inds], (ftx2[inds]).real, )
ax4.set_xlim([-0.25, 0.25]);
ax4.set_xlabel('$t$')

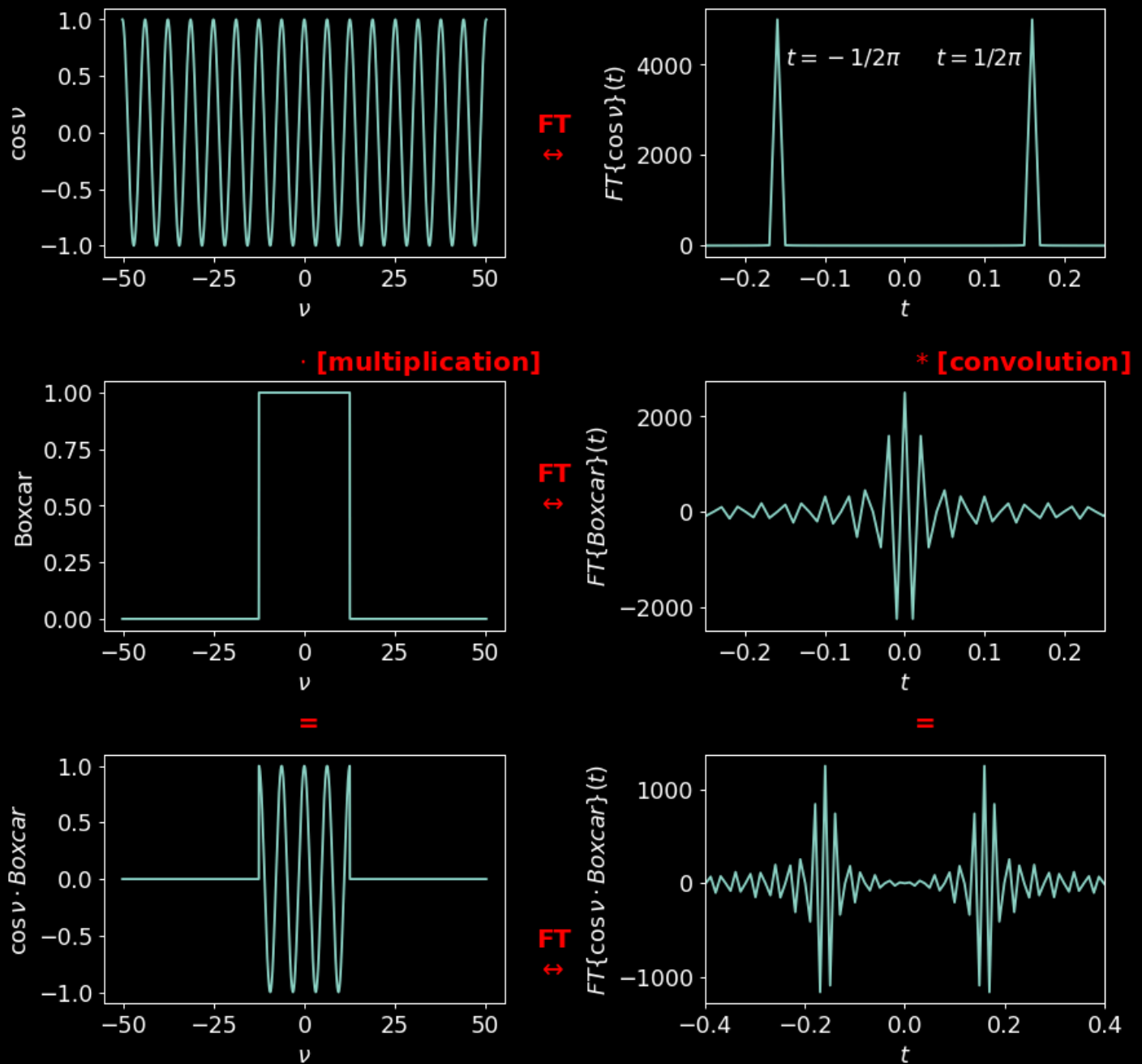
ax4.set_ylabel(r'$FT\{\text{Boxcar}\}(t)$');

ax6 = pl.subplot(3,2,6)
inds = np.argsort(tee)
ftx3 = np.fft.fft(trunc*y)
ax6.plot(tee[inds], (ftx3[inds]).real, )
ax6.set_xlim([-0.4, 0.4]);
ax6.set_ylabel(r'$FT\{\cos \nu \cdot \text{Boxcar}\}(t)$');
ax6.set_xlabel('$t$')

pl.suptitle(r"Use convolution to calculate truncated  $\cos \nu \leftrightarrow FT\{\mathit{Boxcar}\}(t)$ ");
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.5, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.1, "FT\n $\leftrightarrow$ ", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.62, "$\cdot$ [multiplication]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)

```

Use convolution to calculate truncated $\cos v \leftrightarrow FT\{\text{truncated } \cos v\}$

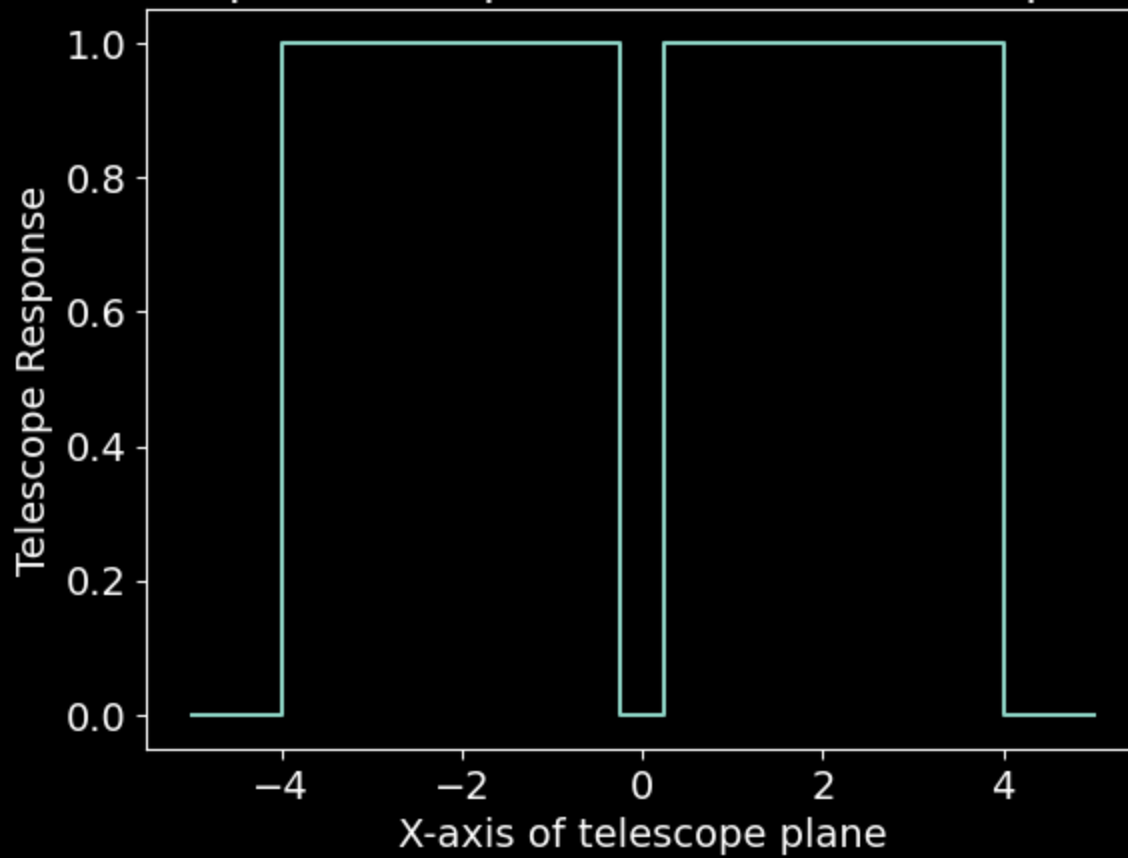


We can do the same for any function.

What if we want to model a telescope aperture with a hole in the middle? (the "hole" might be caused by the secondary mirror)

```
In [38]: pl.title("Simple telescope model - a 'double tophat'")
pl.plot([-5,-4,-4,-0.25,-0.25,0.25,0.25,4,4,5],
        [0, 0, 1, 1, 0, 0, 1,1,0,0])
pl.xlabel("X-axis of telescope plane")
pl.ylabel("Telescope Response");
```

Simple telescope model - a 'double tophat'



```
In [45]: nu = np.linspace(-10*np.pi, 10*np.pi, 10000)
y = ((nu > -3.5*np.pi) & (nu < 3.5*np.pi))# | ((nu > np.pi) & (nu < 2*np.pi))
ftx = np.fft.fft(y)
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])

deltas = np.zeros_like(y)
deltas[np.argmax(np.abs(nu-4*np.pi))] = 1
deltas[np.argmax(np.abs(nu+4*np.pi))] = 1

fig = pl.figure(figsize=(10,10), dpi=90)
ax1 = pl.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'Tophat')

ax2 = pl.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], (ftx[inds]).real)
#ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
#ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-1, 1]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{Tophat\}(t)$'
               '\n$=\mathrm{sinc}(t)$');

ax3 = pl.subplot(3,2,3)
ax3.plot(nu, deltas)
ax3.set_xlabel('$\nu$')
ax3.set_ylabel(r'Double Delta');

ax5 = pl.subplot(3,2,5)
```



```

ax5.plot(nu,
         np.fft.fftshift(np.fft.fft(np.fft.fft(np.fft.fftshift(deltas)) * np.fft.fft(np.
         )
ax5.set_xlabel('$\\nu$')
ax5.set_ylabel(r'$Double Delta * Tophat$');

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(np.fft.fftshift(deltas))
ax4.plot(tee[inds], (ftx2[inds]).real )
ax4.set_xlim([-1, 1]);
ax4.set_xlabel('$t$')

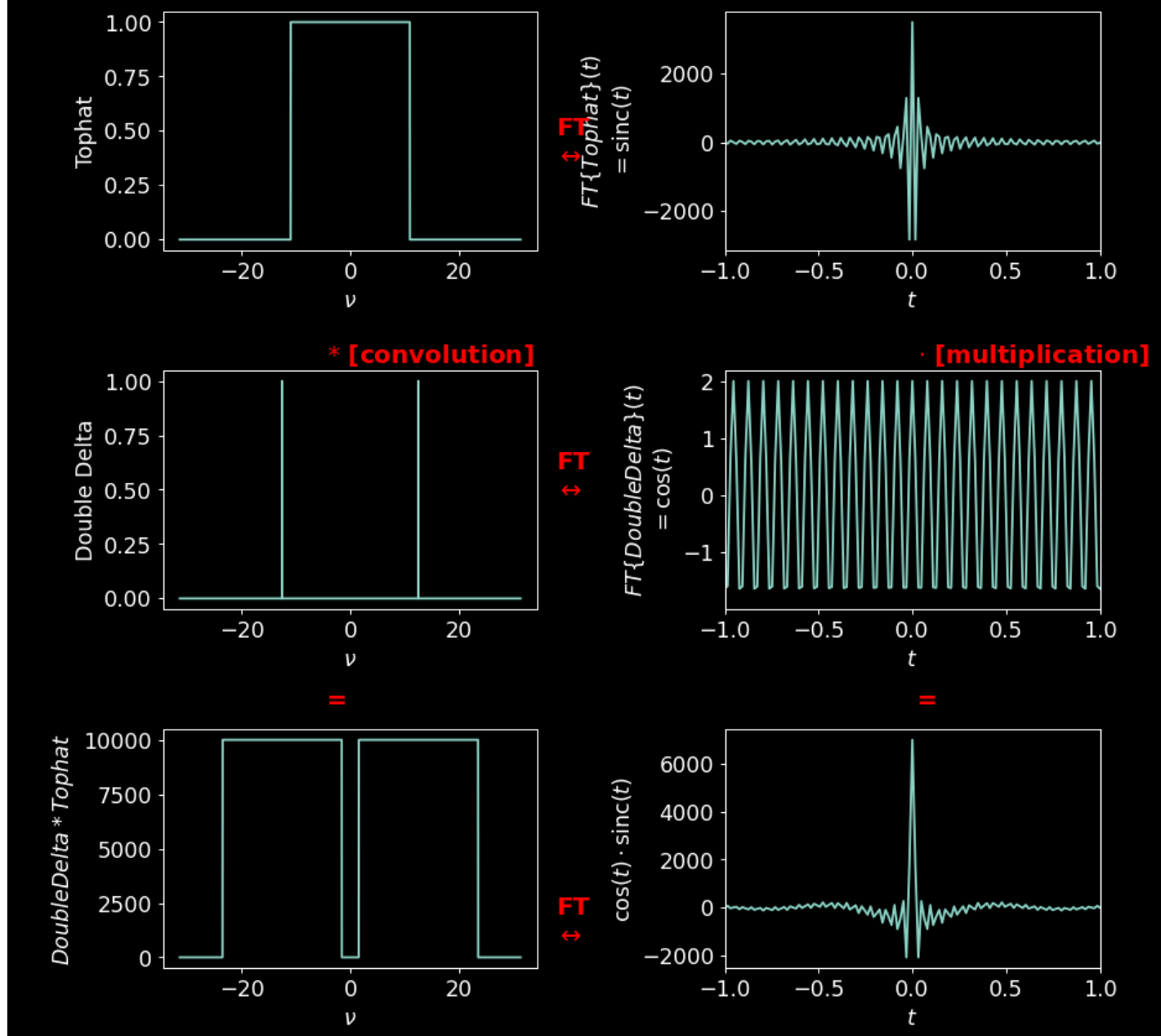
ax4.set_ylabel(r'$FT\{Double Delta\}(t)$'
              '\n$=\cos(t)$');

ax6 = pl.subplot(3,2,6)
inds = np.argsort(tee)
ftx3 = np.fft.fft(np.fft.fftshift(deltas))*np.fft.fft(np.fft.fftshift(y))
ax6.plot(tee[inds], (ftx3[inds]).real, )
ax6.set_xlim([-1, 1]);
ax6.set_ylabel(r'$\cos(t) \cdot \mathrm{sinc}(t)$');
ax6.set_xlabel('$t$')

pl.suptitle(r"Use convolution to calculate the response of two apertures (or a telescope
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weig
pl.text(0.475, 0.5, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weig
pl.text(0.475, 0.1, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weig
pl.text(0.27, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='b
pl.text(0.8, 0.62, "$\cdot$ [multiplication]", transform=fig.transFigure, color='r', w
pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=1
pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16

```

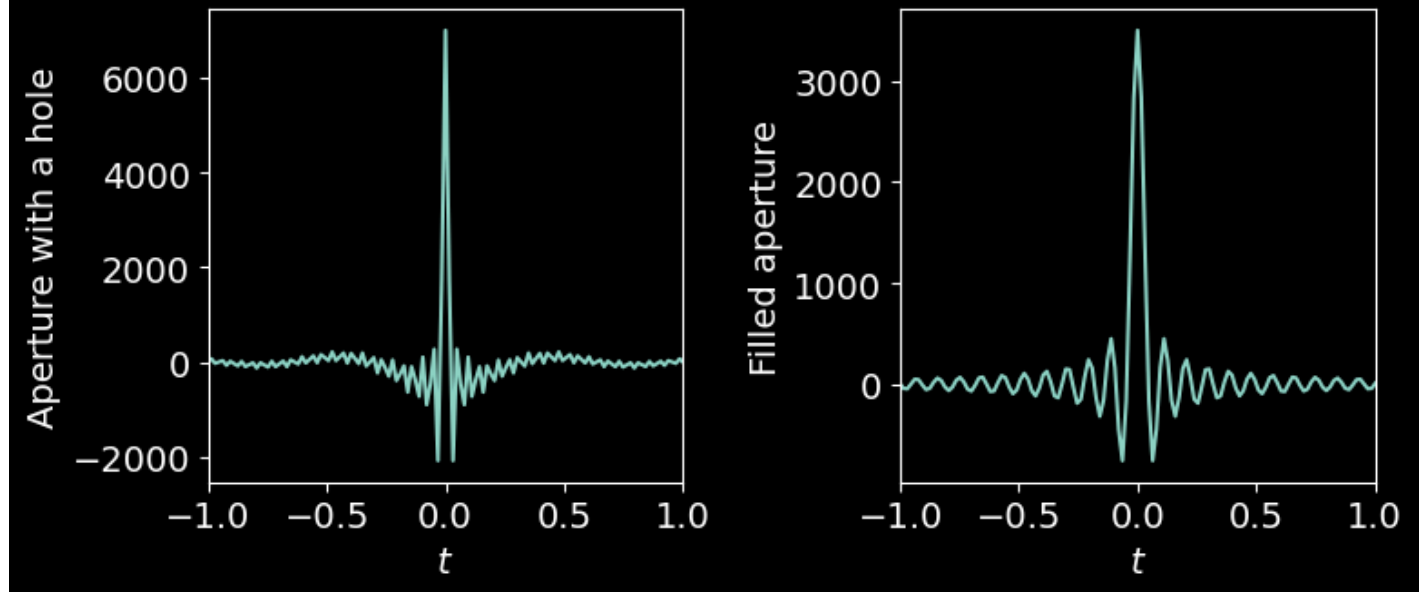
Use convolution to calculate the response of two apertures (or a telescope with a hole)



```
In [40]: pl.figure(figsize=(8,4))
ax1 = pl.subplot(1,2,1)
inds = np.argsort(tee)
ftx3 = np.fft.fft(np.fft.fftshift(deltas))*np.fft.fft(np.fft.fftshift(y))
ax1.plot(tee[inds], (ftx3[inds]).real, )
ax1.set_xlim([-1, 1]);
ax1.set_ylabel('Aperture with a hole');
ax1.set_xlabel('$t$')

ax2 = pl.subplot(1,2,2)
ax2.plot(tee[inds], (np.fft.fft(np.fft.fftshift(y))[inds]).real, )
ax2.set_xlim([-1, 1]);
ax2.set_ylabel('Filled aperture');
ax2.set_xlabel('$t$')
pl.suptitle("Compare: FT{holed aperture} vs FT{filled aperture}")
pl.tight_layout();
```

Compare: FT{holed aperture} vs FT{filled aperture}



One of the most important use cases of the convolution theorem is the *sampling theorem*.

What happens if we discretize our measurement? i.e., we only take samples at regular intervals?

We model this with the shah function

```
In [43]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.exp(-nu**2/(2*(4*np.pi)**2))
ftx = np.fft.fft(y) * dnu
tee = np.fft.fftfreq(y.size) / (nu[1]-nu[0])
fig = pl.figure(figsize=(10,10), dpi=90)
ax1 = pl.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'Smooth (Gaussian) signal')

ax2 = pl.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
#ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
#ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.5, 0.5]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{smooth\ signal\}(t)$');

ax3 = pl.subplot(3,2,3)
deltas = np.zeros_like(y)
deltas[::500] = 1
ax3.plot(nu, deltas)
ax3.set_xlabel('$\nu$')
ax3.set_ylabel(r'shah');

ax5 = pl.subplot(3,2,5)
ax5.plot(nu, deltas * y)
ax5.set_xlabel('$\nu$')
ax5.set_ylabel(r'shah $\cdot$ y');
```

```

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(np.fft.fftshift(deltas)) * dnu
ax4.plot(tee[inds], (ftx2[inds]).real )
ax4.set_xlim([-0.5, 0.5]);
ax4.set_xlabel('$t$')

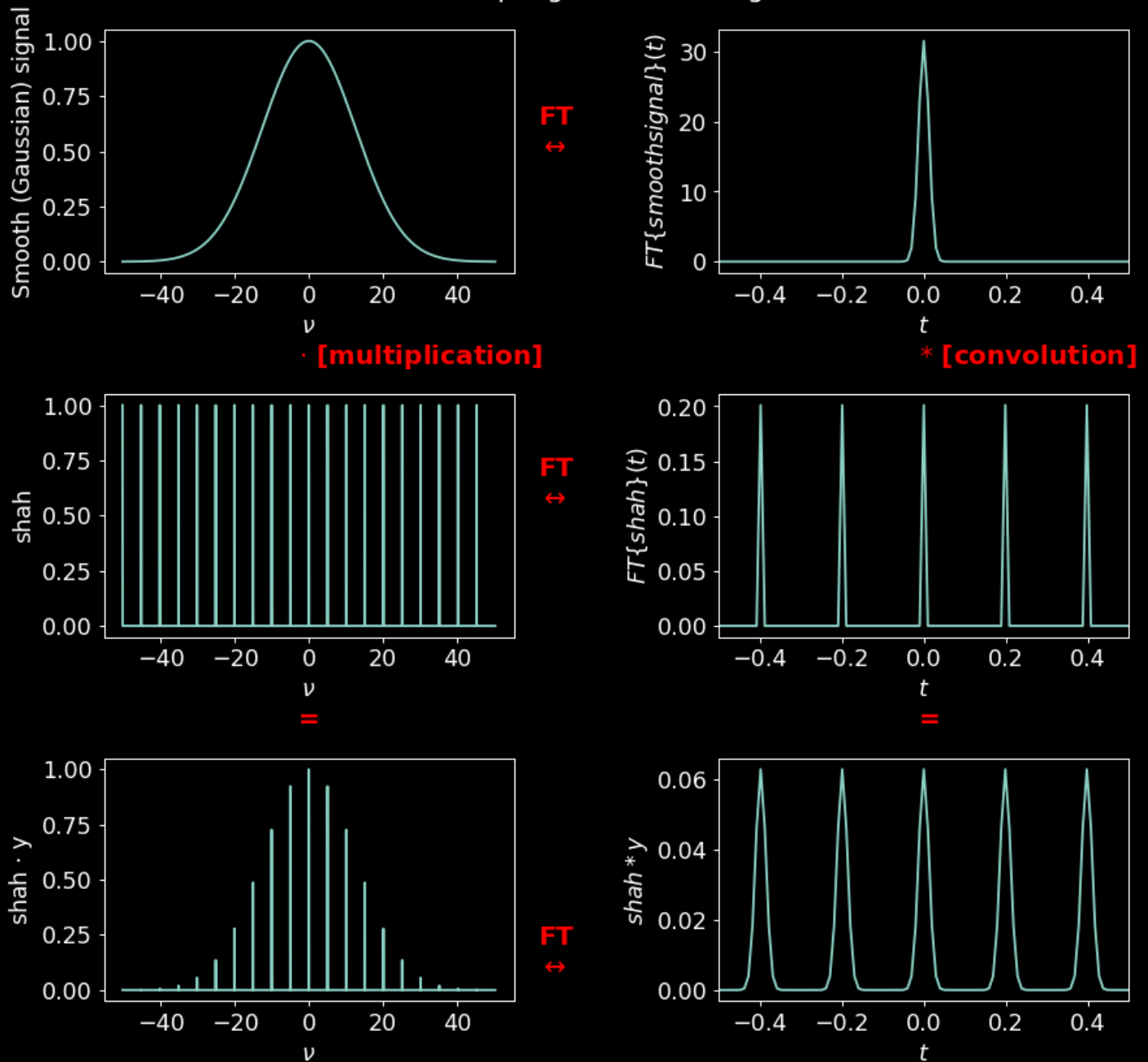
ax4.set_ylabel(r'$FT\{\text{shah}\}(t)$');

ax6 = pl.subplot(3,2,6)
inds = np.argsort(tee)
ftx3 = np.fft.fft(np.fft.fftshift(deltas*y)) * dnu
ax6.plot(tee[inds], np.abs(ftx3[inds]).real, )
ax6.set_xlim([-0.5, 0.5]);
ax6.set_ylabel(r'$\text{shah} * y$');
ax6.set_xlabel('$t$')

pl.suptitle("Use convolution to calculate the effect of sampling:\nFine sampling $\rightarrow$")
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.5, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.1, "FT\n$\rightarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.62, r"$\cdot$ [multiplication]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)

```

Use convolution to calculate the effect of sampling: Fine sampling $\rightarrow$ no blurring



```
In [77]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.exp(-nu**2/(2*(1.0*np.pi)**2))
dnu = nu[1] - nu[0]
ftx = np.fft.fft(y) * dnu
tee = np.fft.fftfreq(y.size) / dnu
fig = pl.figure(figsize=(10,10), dpi=90)
ax1 = pl.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'Smooth (Gaussian) signal')

ax2 = pl.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
#ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
#ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.5, 0.5]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\text{smooth~signal}\}(t)$');
```

```

ax3 = pl.subplot(3,2,3)
deltas = np.zeros_like(y)
deltas[::500] = 1
ax3.plot(nu, deltas)
ax3.set_xlabel('$\\nu$')
ax3.set_ylabel(r'shah');

```

```

ax5 = pl.subplot(3,2,5)
ax5.plot(nu, deltas * y)
ax5.set_xlabel('$\\nu$')
ax5.set_ylabel(r'shah $\\cdot$ y');

```

```

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(np.fft.fftshift(deltas)) * dnu
ax4.plot(tee[inds], (ftx2[inds]).real )
ax4.set_xlim([-0.5, 0.5]);
ax4.set_xlabel('$t$')

```

```

ax4.set_ylabel(r'$FT\{shah\}(t)$');

```

```

ax6 = pl.subplot(3,2,6)
inds = np.argsort(tee)
ftx3 = np.fft.fft(np.fft.fftshift(deltas*y)) * dnu
ax6.plot(tee[inds], (ftx3[inds]).real, )
ax6.set_xlim([-0.5, 0.5]);
ax6.set_ylim(0,ax6.get_ylim()[1])
ax6.set_ylabel(r'$shah * y$');
ax6.set_xlabel('$t$')

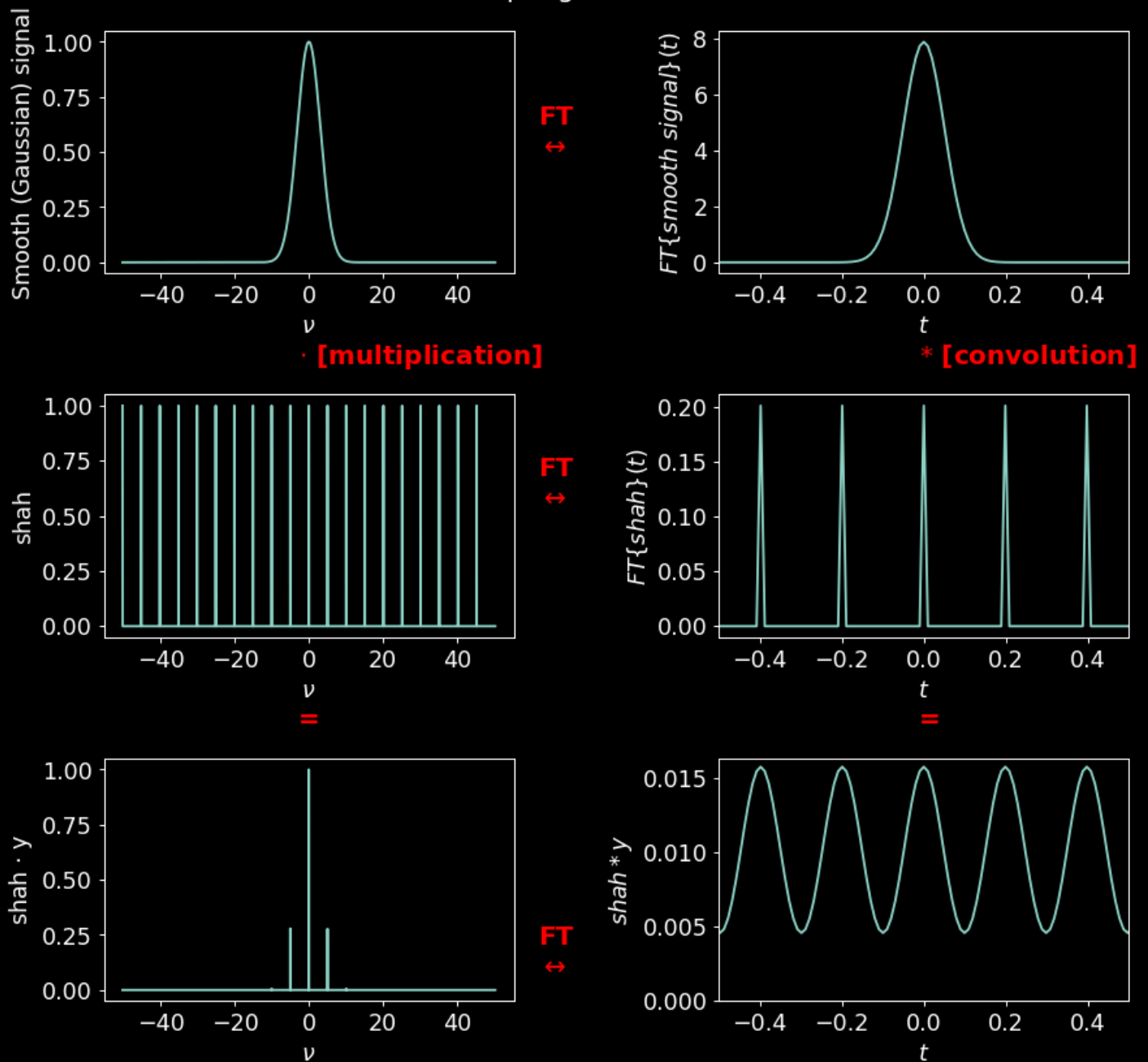
```

```

pl.suptitle("Use convolution to calculate the effect of sampling\nCoarse sampling$\\right$")
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.5, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.1, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.62, "$\\cdot$ [multiplication]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)

```

Use convolution to calculate the effect of sampling Coarse sampling → blended lines



```
In [80]: nu = np.linspace(-16*np.pi, 16*np.pi, 10000)
y = np.exp(-nu**2/(2*(1.0*np.pi)**2))
dnu = nu[1] - nu[0]
ftx = np.fft.fft(y) * dnu
tee = np.fft.fftfreq(y.size) / dnu
fig = pl.figure(figsize=(10,10), dpi=90)
ax1 = pl.subplot(3,2,1)
ax1.plot(nu, y)
ax1.set_xlabel(r'$\nu$')
ax1.set_ylabel(r'Smooth (Gaussian) signal')

ax2 = pl.subplot(3,2,2)
inds = np.argsort(tee)
ax2.plot(tee[inds], np.abs(ftx[inds]))
#ax2.text(-1/(2*np.pi) + 0.01, 4000, r'$t=-1/2\pi$')
#ax2.text(1/(2*np.pi) - 0.01, 4000, r'$t=1/2\pi$', ha='right')
ax2.set_xlim([-0.5, 0.5]);
ax2.set_xlabel('$t$')
ax2.set_ylabel(r'$FT\{\text{smooth~signal}\}(t)$');
```

```

ax3 = pl.subplot(3,2,3)
deltas = np.zeros_like(y)
deltas[::1000] = 1
ax3.plot(nu, deltas)
ax3.set_xlabel('$\\nu$')
ax3.set_ylabel(r'shah');

```

```

ax5 = pl.subplot(3,2,5)
ax5.plot(nu, deltas * y)
ax5.set_xlabel('$\\nu$')
ax5.set_ylabel(r'shah  $\cdot$  y');

```

```

ax4 = pl.subplot(3,2,4)
inds = np.argsort(tee)
ftx2 = np.fft.fft(np.fft.fftshift(deltas)) * dnu
ax4.plot(tee[inds], (ftx2[inds]).real )
ax4.set_xlim([-0.5, 0.5]);
ax4.set_xlabel('$t$')

```

```

ax4.set_ylabel(r'$FT\{shah\}(t)$');

```

```

ax6 = pl.subplot(3,2,6)
inds = np.argsort(tee)
ftx3 = np.fft.fft(np.fft.fftshift(deltas*y)) * dnu
ax6.plot(tee[inds], (ftx3[inds]).real, )
ax6.set_xlim([-0.5, 0.5]);
ax6.set_ylim(0,ax6.get_ylim()[1]*1.5)
ax6.set_ylabel(r'$shah * y$');
ax6.set_xlabel('$t$')

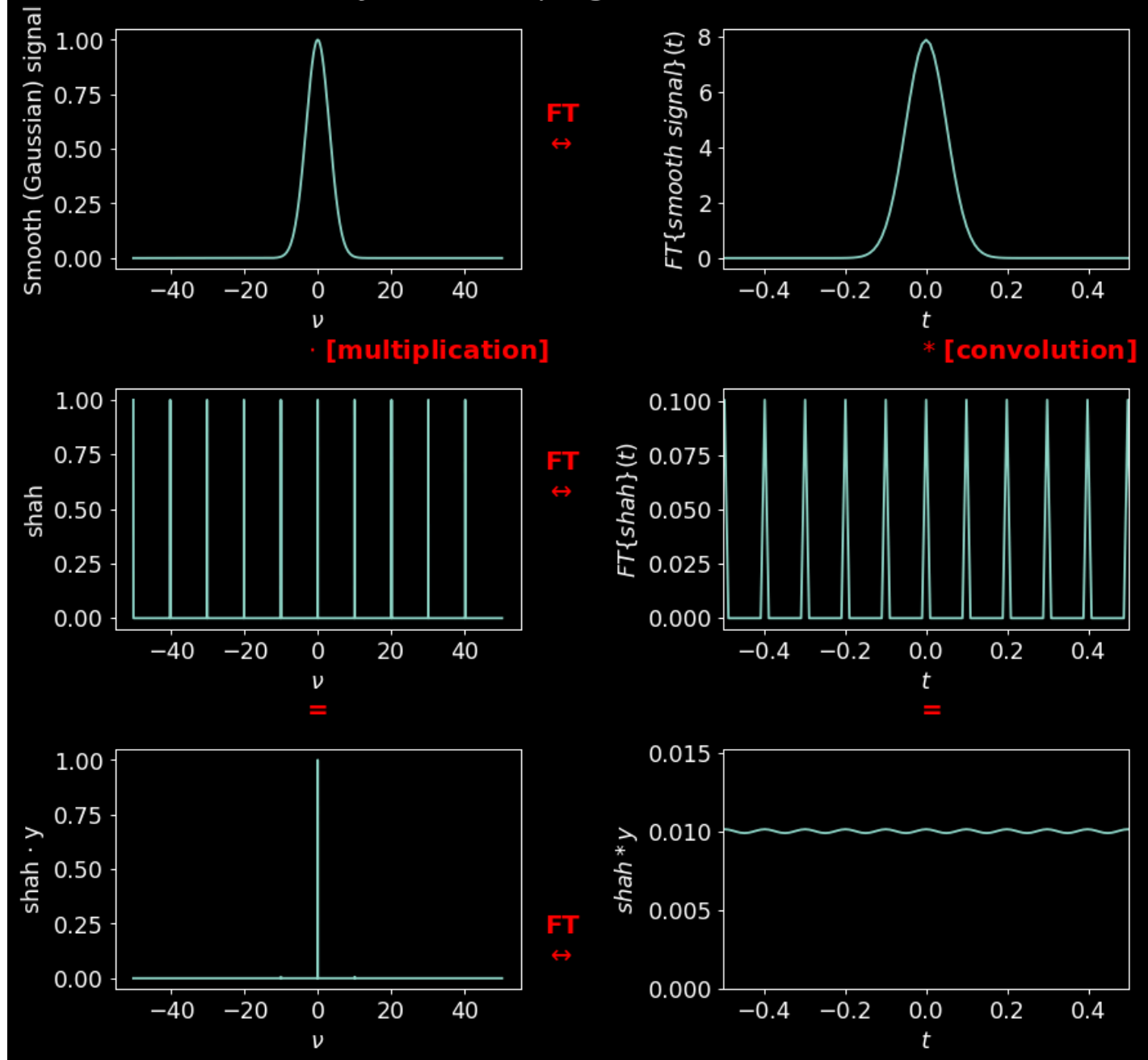
```

```

pl.suptitle("Use convolution to calculate the effect of sampling\nVery coarse sampling$")
pl.tight_layout()
pl.subplots_adjust(wspace=0.5, hspace=0.5)
pl.text(0.475, 0.8, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.5, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.475, 0.1, "FT\\n$\\leftarrow$", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.62, "$\\cdot$ [multiplication]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.62, "$*$ [convolution]", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.27, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)
pl.text(0.8, 0.31, "=", transform=fig.transFigure, color='r', weight='bold', fontsize=16)

```


Use convolution to calculate the effect of sampling Very coarse sampling → no information



The requirement for adequate sampling is the Nyquist criterion:

$$\frac{1}{\Delta t} \geq 2f_{max}$$

i.e., the sampling frequency has to be at least *twice* the highest frequency in the signal.

The Nyquist sampling *rate* is

$$\delta T = \frac{1}{2f_{max}}$$

If the Nyquist requirement is not met, aliasing of higher frequencies to lower frequencies will occur.

Examples of aliasing:

- Wagon wheels appear to roll backwards in old western movies.

- Car wheels appear to roll backwards when viewed at night under a streetlamp.
- High frequency noise in a timestream (if you are not careful to bandwidth-limit the signal with a low pass filter that rolls off the timestream signal at less than half the sampling frequency)

Exercises

Without formally evaluating the Fourier integral, draw by hand the functions below on the left side of the page, and their Fourier transforms on the right side of the page. (Note: t_0 and f_0 indicate constants, not variables.)

1. $A \cos(2f_0 t)$
2. $A \sin(2f_0 t)$
3. $\text{sinc}(2ft_0)$
4. $\text{sinc}^2(2ft_0)$
5. $\cos(4 \cdot ft_0) \text{sinc}(2ft_0)$
6. $\cos(2 \cdot ft_0) \text{sinc}(2ft_0)$
7. $e^{-t^2} \cos(2f_0 t)$